Extending Multibody Simulation: Closed-Form Dynamics from URDF Models for Model-Based Control and Trajectory Optimization of Serial Manipulators

Ahmed Askar[1,2]

[1]Department of Mechatronics and Robotics Engineering, School of Innovative Design Engineering, Egypt-Japan University of Science and Technology (E-JUST), New Borg El-Arab City, Alexandria, Egypt
[2]Department of Production Engineering and Mechanical Design, Faculty of Engineering, Mansoura University, Mansoura, Egypt

October 29, 2026

Abstract

Multibody simulators based on the Unified Robot Description Format (URDF) predict robot motion accurately but keep their equations of motion implicit, which limits model-based control and trajectory optimization. We present an automated pipeline that extends such simulators with closed-form, differentiable dynamics generated directly from URDF models of serial manipulators. The pipeline synthesizes Denavit–Hartenberg parameters, resolving frame ambiguities, projects corrupted CAD inertia tensors onto the physically realizable set, and computes Coriolis and centrifugal terms as a directional derivative of the generalized momentum, with cost bounded by a constant multiple of the mass-matrix cost. Forward dynamics uses symbolic Cholesky factorization instead of matrix inversion. Across chains of 3 to 14 degrees of freedom, graph size scales as $\mathcal{O}(n^{2.29})$ rather than $\mathcal{O}(n^{3.25})$ for Christoffel expansions, cutting forward-dynamics instructions by 55.4% for a 6-DOF arm and 75.5% at 14 DOF. Models of four industrial manipulators agree with MuJoCo and Simscape, and compiled C forward dynamics runs in 78.7 μs. The exact derivatives enable constrained trajectory optimization, converging on all 15 test problems per robot (100% convergence in 7.9–15.6 iterations, reducing iterations by 95.6%–96.8% and accelerating solve time up to 14.39× over Quasi-Newton L-BFGS), and computed-torque control that maintains robust tracking under moderate parameter mismatch, while large unmodeled payloads motivate adaptive parameter calibration.

Introduction

Closed-form kinematic and dynamic models remain the cornerstone of robot analysis, nonlinear control, and robotics education. Minimal parameterizations, most notably the standard Denavit–Hartenberg (DH) convention [1–4], combined with Euler–Lagrange mechanics, establish an explicit relationship between kinematic geometry, inertial properties, and system behavior. This analytical transparency provides the structural insight required for dynamic analysis, formal stability assessment, and systematic model-based controller synthesis. In contemporary robotics practice, however, development is increasingly driven by software ecosystems such as ROS [5], MoveIt [6], Gazebo [7], PyBullet [8], MuJoCo [9], and Simscape Multibody [10]. These platforms predominantly describe articulated mechanisms using the Unified Robot Description Format (URDF) [11, 12], which represents robots as trees of rigid bodies connected through local coordinate transformations. Although highly effective for software integration, motion planning, collision detection, and numerical simulation, this representation differs fundamentally from the minimal analytical parameterizations employed in classical robot modeling. As a result, a persistent gap has emerged between simulation-oriented robot descriptions and the explicit analytical models required for rigorous dynamics analysis and control design [13, 14].

This gap is further widened by the computational paradigms underlying modern simulation engines. Rather than deriving explicit symbolic equations of motion, contemporary frameworks typically employ constrained optimization techniques [9] or recursive rigid-body algorithms, most notably Featherstone's Articulated Body Algorithm (ABA) and Recursive Newton–Euler Algorithm (RNEA) [15, 16]. By exploiting the tree structure of articulated mechanisms, these methods enable highly efficient forward and inverse dynamics computations while supporting robust contact and collision handling. Their efficiency, however, comes from avoiding the construction of global analytical models. Consequently, users interact with the simulator primarily through numerical state-transition operators that map joint positions, velocities, and actuation inputs to accelerations, whereas the

underlying dynamic relationships remain embedded within the computational framework and are not exposed in an explicit analytical form. This abstraction is advantageous for simulation and real-time execution but limits direct access to the mathematical structure required for theoretical analysis, stability proofs, and model-based controller design.

The absence of explicit dynamic equations poses a significant challenge for model-based control and trajectory optimization, both of which require the equations of motion and their derivatives. High-fidelity environments such as MSC ADAMS, Simscape Multibody, Gazebo, and MuJoCo accurately perform dynamic simulation, but they generally do not expose the nonlinear equations required for systematic controller synthesis [17, 18]. Where control-design utilities are available, they typically rely on local Jacobian linearizations around fixed operating points, which limits their validity to a narrow region of operation [17, 18]. Performance therefore deteriorates during highly dynamic maneuvers, in which configuration-dependent inertia variations and velocity-dependent Coriolis and centrifugal effects dominate. Without globally valid analytical models, practitioners often resort to model-free architectures, most commonly decentralized PID controllers, whose inability to compensate for nonlinear dynamic couplings leads to reduced tracking accuracy, increased phase lag, and actuator chattering under demanding conditions [19, 20].

Data-driven model discovery, particularly Sparse Identification of Nonlinear Dynamics (SINDy), has been explored as a way to recover governing equations directly from simulated trajectories [17, 18, 21, 22]. Although successful in low-dimensional systems, these approaches are difficult to extend to articulated manipulators. The trigonometric complexity of multi-degree-of-freedom kinematics produces large candidate libraries and poorly conditioned regression problems, and physically meaningful models typically require extensive excitation trajectories, library reduction strategies, and physics-constrained optimization [17, 18]. These difficulties motivate methods that derive exact and physically consistent equations of motion directly from CAD- and URDF-based robot descriptions, instead of reconstructing them indirectly from simulated data.

The demand for explicit analytical models is further amplified by the emergence of industrial digital twins [13, 14, 23]. A URDF model captures only the nominal geometry and inertial properties of a robot, and it cannot represent the manufacturing tolerances, assembly deviations, payload variations, or structural compliance encountered in deployed hardware [19, 24]. Bridging this simulation-to-reality gap requires calibrating kinematic and dynamic parameters from experimental measurements [21, 23]. Such calibration is most naturally formulated within gradient-based optimization frameworks, which in turn require differentiable analytical models that can be embedded in tools such as CasADi [25].

Existing solutions address only part of this need. Available URDF conversion tools either rely on auxiliary dummy frames, fail on general branched trees, or stop at kinematics without propagating inertial properties into the synthesized DH frames [26–28]. A general framework that transforms industrial CAD and URDF descriptions into explicit and physically consistent analytical dynamic models therefore remains largely unavailable [11, 28]. Three technical obstacles explain this gap. First, a URDF joint is an unconstrained six-degree-of-freedom transformation referenced to arbitrary CAD origins, whereas the DH convention imposes two strict geometric rules on the frame assignment and leaves it non-unique whenever consecutive joint axes are parallel or collinear [2, 28, 29]. Second, link masses, centers of mass, and inertia tensors are defined relative to CAD link origins and must be transformed exactly into the DH frames; moreover, CAD-derived and historical inertia tensors frequently violate rigid-body realizability, that is, positive definiteness and the triangle inequality on the principal moments [11, 18, 20], which causes numerical breakdowns in dynamic simulation. Third, deriving closed-form Euler–Lagrange equations with computer algebra suffers from severe expression swell: forming the n^3 Christoffel symbols and symbolically inverting the mass matrix generate expressions that grow rapidly beyond six degrees of freedom and preclude real-time execution. Recursive spatial-vector libraries avoid this swell through procedural C/C++ routines, but they do not expose compressed, DH-consistent symbolic graphs across the MATLAB, Simulink, and Python ecosystems [16].

This paper addresses these obstacles with a deterministic, fully automated framework that converts raw, uncalibrated industrial URDF models into physically consistent, closed-form analytical dynamic models (Figure 1). The aim is to extend multibody simulation, not to replace it: the generated models are verified against established physics engines and operate alongside them, so that a single URDF description serves both as the simulated plant and as the source of the equations needed for model-based control, trajectory optimization, and digital-twin calibration. The framework synthesizes DH-consistent kinematic descriptions, propagates and validates the inertial properties, derives the closed-form Euler–Lagrange equations of motion, and reformulates the dynamic computations through algorithmic differentiation to overcome the scalability limits traditionally associated with symbolic robot dynamics. We test the hypothesis that the combinatorial growth which has restricted closed-form Lagrangian mechanics to low-dimensional manipulators is not intrinsic to analytical mechanics, but an artifact of the Christoffel-tensor formulation, and that removing it reduces the cost of the sensitivity evaluations that dominate gradient-based trajectory optimization, including its receding-horizon form, nonlinear model predictive control (NMPC), and calibration. We focus on serial manipulators, the dominant class of industrial robots and the backbone of manufacturing, assembly, and material-handling tasks; the underlying principles of automated model extraction, inertial parameter propagation, and differentiable symbolic dynamics provide a foundation for

future extension to mobile manipulators, legged systems, humanoid robots, and hybrid kinematic structures.

The main contributions of this paper are as follows:

1. **Automated Kinematic and Inertial Synthesis:** We present a geometric synthesis pipeline that maps arbitrary URDF joint trees to minimal DH parameters through a four-regime classification that resolves gauge ambiguities, together with a projection of link inertia tensors onto the physically realizable set that guarantees rigid-body realizability.

2. **Momentum-Based Coriolis Formulation:** We formulate an exact directional-derivative expression for the Coriolis and centrifugal bias vector as the derivative of the generalized momentum. We establish its equivalence to the classical Christoffel formulation and its preservation of the skew-symmetric passivity property ($\dot{\boldsymbol{M}} - 2\boldsymbol{C}$), demonstrating that its evaluation cost is bounded by a constant multiple of the mass matrix cost.

3. **Forward Dynamics via Symbolic Cholesky Factorization:** Exploiting the guaranteed positive definiteness of the sanitized models, we construct a pivot-free symbolic Cholesky solver that avoids symbolic matrix inversion. Together with the momentum formulation, it reduces the forward dynamics instruction count by 55.4% on a 6-DOF arm and by up to 75.5% on 14-DOF chains, and it reduces the dynamics-sensitivity cost of NMPC by 40.6% to 48.2%.

4. **Multi-Platform Industrial Validation:** We demonstrate the end-to-end framework on four industrial manipulators (UR5, UR16e, ABB IRB 120, and Puma 560), verifying agreement with established physics engines (MuJoCo, Simscape) through an automated five-stage verification procedure, and we show its deployment in computed torque control and in receding-horizon NMPC with obstacle and torque constraints, a trajectory optimization problem solved at every control step, through three software tiers that range from symbolic MATLAB expressions to compiled C code.

The remainder of the paper is organized as follows. The next three sections detail the analytical framework for kinematic and dynamic synthesis, the optimal control and software implementation across three tiers, and the multi-stage verification protocol. The Results section then presents the derivative benchmark, morphological verification, multi-DOF scalability, trajectory optimization, and closed-loop control experiments on physics engines under parametric mismatch. Finally, the Discussion evaluates the framework relative to numerical simulators and recursive algorithms, highlighting practical implications for receding-horizon control and robotics education, followed by concluding remarks.

1 Analytical Framework for Kinematic and Dynamic Modeling of Serial Manipulators

The objective of this section is to transform a raw robot description into a complete, executable dynamic model without requiring any manual derivation. The modeling procedure follows the natural sequence dictated by the underlying physics. A URDF file represents a robot as a tree of links and joints whose frame placements and axis orientations are selected primarily for CAD representation rather than analytical modeling. Therefore, the serial chain of actuated joints is first identified, and its geometry is expressed in the canonical DH convention. Particular attention is given to degenerate configurations, such as parallel or collinear joint axes, for which the convention is not unique. It is further demonstrated that the selected conventions in these cases do not affect the resulting kinematic description. Once the geometric structure has been established, the mass properties of each link are expressed in the corresponding frames and verified for physical consistency, since inertial parameters exported from CAD software are often inconsistent. The corrected inertial data are then combined with the kinematic description to construct the inertial and gravitational components of the equations of motion. Subsequently, the velocity-dependent Coriolis and centrifugal terms are derived from the time derivative of the generalized momentum through algorithmic differentiation. This approach remains mathematically equivalent to the classical formulation while avoiding its computationally expensive symbolic expansion. Finally, joint accelerations are obtained by factorizing the manipulator inertia matrix and solving the resulting system through triangular substitution rather than explicit matrix inversion. The resulting framework provides an efficient, numerically robust, and fully symbolic forward-dynamics model that is generated automatically from the URDF description.

1.1 Kinematic Chain Extraction and Denavit–Hartenberg Parameter Synthesis

Transforming an arbitrary URDF into a valid analytical model begins with topological graph parsing and geometric axis alignment. Given an input URDF XML file, an entity-guarded parser constructs a directed acyclic graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where vertices $\mathcal{V} = \{L_0, \ldots, L_M\}$ denote rigid links and directed edges $\mathcal{E} = \{J_1, \ldots, J_M\}$

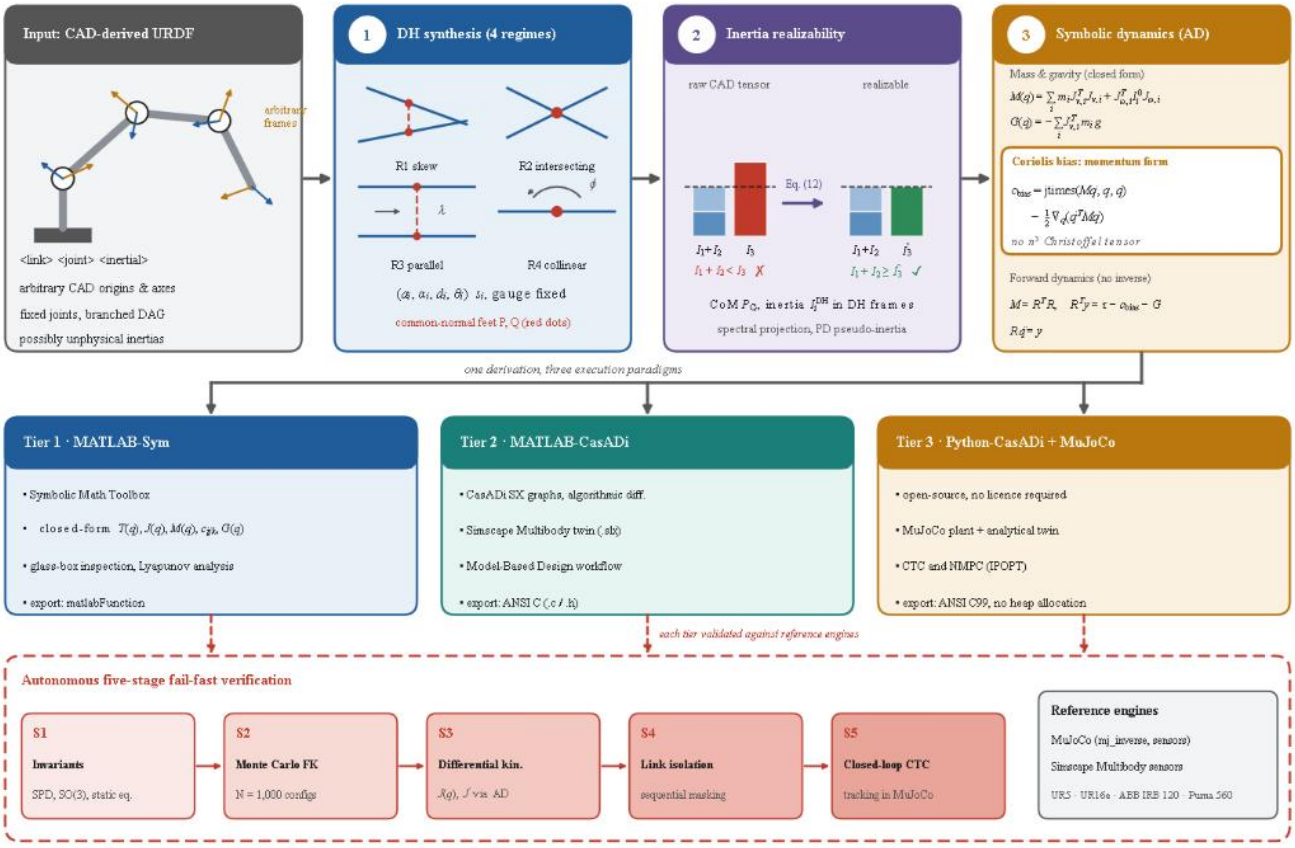


Figure 1: **Three-Tier System Architecture and Computational Pipeline.** Raw CAD-derived URDF descriptions (with arbitrary coordinate frames and unphysical CAD inertias) undergo: **(1) DH Synthesis across 4 Regimes:** resolving continuous gauge freedoms for skew (R1), intersecting (R2), parallel (R3), and collinear (R4) joint axes via common-normal feet P, Q and automated axis-sign detection s_i; **(2) Inertia Realizability Projection:** spectral mapping of raw CAD inertia tensors onto the physically realizable set, enforcing the rigid-body triangle inequality ($I_1 + I_2 \geq I_3$) and positive-definite pseudo-inertias; **(3) Symbolic Dynamics via Algorithmic Differentiation:** closed-form mass metric $\boldsymbol{M}(\boldsymbol{q})$ and gravity $\boldsymbol{G}(\boldsymbol{q})$, generalized momentum directional derivative $\mathbf{c}_{\text{bias}}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \text{jtimes}(\boldsymbol{M}\dot{\boldsymbol{q}}, \boldsymbol{q}, \dot{\boldsymbol{q}}) - \frac{1}{2}\nabla_{\boldsymbol{q}}(\dot{\boldsymbol{q}}^T \boldsymbol{M}\dot{\boldsymbol{q}})$ eliminating n^3 Christoffel tensors, and Cholesky forward dynamics without explicit matrix inversion ($\boldsymbol{M} = \boldsymbol{R}^T\boldsymbol{R}$). The unified derivation branches into three complementary execution tiers: **Tier 1 (`MATLAB-Sym`)** for glass-box algebraic inspectability and Lyapunov analysis; **Tier 2 (`MATLAB-CasADi`)** for Simscape Multibody Model-Based Design; and **Tier 3 (`Python-MuJoCo`)** for open-source nonlinear control (CTC and NMPC under IPOPT) with bare-metal ANSI C99 export. All tiers are validated against a **Five-Stage Verification Procedure** (S1: Physical invariants; S2: Monte Carlo forward kinematics; S3: Differential kinematics $\boldsymbol{J}, \dot{\boldsymbol{J}}$; S4: Sequential link inertia/gravity isolation; S5: Closed-loop CTC tracking in MuJoCo) cross-referenced against industrial platforms.

represent kinematic joints defined by parent-to-child homogeneous transformations $\boldsymbol{T}_{\text{joint},k}(\boldsymbol{t}_k, \boldsymbol{\varphi}_k) \in SE(3)$ and local actuation unit axes $\hat{\boldsymbol{u}}_k \in S^2$ [28].

The structural base link L_0 is isolated via set difference between parent and child vertex sets. A breadth-first search traversing from L_0 extracts the ordered active serial chain $\mathcal{K} = \{(L_0, J_1, L_1), \ldots, (L_{n-1}, J_n, L_n)\}$ containing n actuated joints with joint type indicators $\sigma_i \in \{1 \text{ (revolute)}, 0 \text{ (prismatic)}\}$. Intermediate non-actuated connections (type = fixed) along the chain are topologically contracted: their relative homogeneous transformations and inertial tensors are folded into the nearest upstream active link via rigid-body parallel-axis transformations [28].

At the home configuration ($\boldsymbol{q} = \boldsymbol{0}$), let $\boldsymbol{O}_{i-1}, \boldsymbol{O}_i \in \mathbb{R}^3$ denote the global origins of consecutive active joints $i-1$ and i, and let $\hat{\boldsymbol{z}}_{i-1}, \hat{\boldsymbol{z}}_i \in S^2$ denote their unit joint axes expressed in base coordinates $\{\mathcal{F}_0\}$. Defining the origin displacement $\Delta\boldsymbol{o}_i = \boldsymbol{O}_i - \boldsymbol{O}_{i-1}$, the cross-product magnitude $\delta_\times = \|\hat{\boldsymbol{z}}_{i-1} \times \hat{\boldsymbol{z}}_i\|_2$, and the normal separation distance yields the geometric classification metrics [28]:

$$d_{\min} = \frac{|\Delta\boldsymbol{o}_i \cdot (\hat{\boldsymbol{z}}_{i-1} \times \hat{\boldsymbol{z}}_i)|}{\delta_\times} \quad (\delta_\times > \epsilon_\parallel), \qquad d_\perp = \|\Delta\boldsymbol{o}_i - (\Delta\boldsymbol{o}_i \cdot \hat{\boldsymbol{z}}_{i-1})\hat{\boldsymbol{z}}_{i-1}\|_2 \quad (\delta_\times \leq \epsilon_\parallel). \tag{1}$$

Using numerical thresholds $\epsilon_\parallel = 10^{-5}$ and $\epsilon_\cap = 10^{-6}$ m, each consecutive axis pair $(\hat{\boldsymbol{z}}_{i-1}, \hat{\boldsymbol{z}}_i)$ is classified into

one of four canonical geometric regimes to compute the foot-points $\boldsymbol{P}_{i-1} \in \mathcal{L}_{i-1}(\boldsymbol{O}_{i-1}, \hat{\boldsymbol{z}}_{i-1})$ and $\boldsymbol{Q}_i \in \mathcal{L}_i(\boldsymbol{O}_i, \hat{\boldsymbol{z}}_i)$ defining the common normal line segment:

1. **Regime 1 (General Skew Axes, $\delta_\times > \epsilon_\parallel$, $d_{\min} > \epsilon_\cap$):** The spatial lines $\mathcal{L}_{i-1}(s) = \boldsymbol{O}_{i-1} + s\hat{\boldsymbol{z}}_{i-1}$ and $\mathcal{L}_i(t) = \boldsymbol{O}_i + t\hat{\boldsymbol{z}}_i$ admit a unique shortest perpendicular with no gauge freedom. Minimizing the squared Euclidean distance $\|\mathcal{L}_{i-1}(s) - \mathcal{L}_i(t)\|_2^2$ yields the unconstrained 2×2 linear system:

$$\begin{bmatrix} s^* \\ t^* \end{bmatrix} = \frac{1}{1 - (\hat{\boldsymbol{z}}_{i-1} \cdot \hat{\boldsymbol{z}}_i)^2} \begin{bmatrix} 1 & \hat{\boldsymbol{z}}_{i-1} \cdot \hat{\boldsymbol{z}}_i \\ \hat{\boldsymbol{z}}_{i-1} \cdot \hat{\boldsymbol{z}}_i & 1 \end{bmatrix} \begin{bmatrix} \Delta\boldsymbol{o}_i \cdot \hat{\boldsymbol{z}}_{i-1} \\ -\Delta\boldsymbol{o}_i \cdot \hat{\boldsymbol{z}}_i \end{bmatrix}, \tag{2}$$

   providing the foot-points $\boldsymbol{P}_{i-1} = \boldsymbol{O}_{i-1} + s^*\hat{\boldsymbol{z}}_{i-1}$ and $\boldsymbol{Q}_i = \boldsymbol{O}_i + t^*\hat{\boldsymbol{z}}_i$, with normal axis $\hat{\boldsymbol{x}}_i = (\boldsymbol{Q}_i - \boldsymbol{P}_{i-1})/\|\boldsymbol{Q}_i - \boldsymbol{P}_{i-1}\|_2$.

2. **Regime 2 (Intersecting Axes, $\delta_\times > \epsilon_\parallel$, $d_{\min} \le \epsilon_\cap$):** The axes intersect at a single physical point $\boldsymbol{P}_{i-1} = \boldsymbol{Q}_i$ (enforcing link length $a_i = 0$) computed via Eq. (2). The common normal direction is uniquely determined by $\hat{\boldsymbol{x}}_i = (\hat{\boldsymbol{z}}_{i-1} \times \hat{\boldsymbol{z}}_i)/\|\hat{\boldsymbol{z}}_{i-1} \times \hat{\boldsymbol{z}}_i\|_2$.

3. **Regime 3 (Parallel Non-Collinear Axes, $\delta_\times \le \epsilon_\parallel$, $d_\perp > \epsilon_\cap$):** Because the axes are parallel, Eq. (2) becomes singular due to a continuous translational gauge freedom along $\hat{\boldsymbol{z}}_{i-1}$. The canonical solver resolves this ambiguity by anchoring $s^* = \lambda_i$ (defaulting to $\lambda_i = 0$, setting $\boldsymbol{P}_{i-1} = \boldsymbol{O}_{i-1}$), projecting orthogonally onto $\mathcal{L}_i$ via $t^* = (\boldsymbol{P}_{i-1} - \boldsymbol{O}_i) \cdot \hat{\boldsymbol{z}}_i$, setting $\boldsymbol{Q}_i = \boldsymbol{O}_i + t^*\hat{\boldsymbol{z}}_i$, and defining $\hat{\boldsymbol{x}}_i = (\boldsymbol{Q}_i - \boldsymbol{P}_{i-1})/\|\boldsymbol{Q}_i - \boldsymbol{P}_{i-1}\|_2$.

4. **Regime 4 (Collinear Axes, $\delta_\times \le \epsilon_\parallel$, $d_\perp \le \epsilon_\cap$):** The joint axes coincide ($a_i = 0$, $\alpha_i \in \{0, \pi\}$), admitting both translational gauge freedom $\lambda_i \in \mathbb{R}$ and rotational gauge freedom $\phi_i \in [-\pi, \pi)$ about $\hat{\boldsymbol{z}}_{i-1}$. The canonical solver sets $\boldsymbol{P}_{i-1} = \boldsymbol{Q}_i = \boldsymbol{O}_{i-1}$ and aligns the normal orientation with the preceding frame via $\hat{\boldsymbol{x}}_i = \mathrm{Rot}_z(\phi_i)\hat{\boldsymbol{x}}_{i-1}$ (with default canonical choice $\phi_i = 0$).

From the synthesized orthonormal frames $\boldsymbol{R}_i^{\mathrm{DH}} = [\hat{\boldsymbol{x}}_i,\ \hat{\boldsymbol{z}}_i \times \hat{\boldsymbol{x}}_i,\ \hat{\boldsymbol{z}}_i] \in SO(3)$ and origins $\boldsymbol{O}_i^{\mathrm{DH}} = \boldsymbol{Q}_i$, the four classical DH parameters are extracted analytically:

$$a_i = (\boldsymbol{Q}_i - \boldsymbol{P}_{i-1}) \cdot \hat{\boldsymbol{x}}_i, \qquad \alpha_i = \mathrm{atan2}((\hat{\boldsymbol{z}}_{i-1} \times \hat{\boldsymbol{z}}_i) \cdot \hat{\boldsymbol{x}}_i,\ \hat{\boldsymbol{z}}_{i-1} \cdot \hat{\boldsymbol{z}}_i), \tag{3}$$

$$d_i = (\boldsymbol{P}_{i-1} - \boldsymbol{O}_{i-1}^{\mathrm{DH}}) \cdot \hat{\boldsymbol{z}}_{i-1}, \qquad \theta_i^{\mathrm{off}} = \mathrm{atan2}((\hat{\boldsymbol{x}}_{i-1} \times \hat{\boldsymbol{x}}_i) \cdot \hat{\boldsymbol{z}}_{i-1},\ \hat{\boldsymbol{x}}_{i-1} \cdot \hat{\boldsymbol{x}}_i). \tag{4}$$

These parameters yield the standard link transformation matrix $\boldsymbol{A}_i(q_i) = \mathrm{Rot}_z(\theta_i)\,\mathrm{Trans}_z(d_i)\,\mathrm{Trans}_x(a_i)\,\mathrm{Rot}_x(\alpha_i) \in SE(3)$:

$$\boldsymbol{A}_i(q_i) = \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{5}$$

where $\theta_i = \theta_i^{\mathrm{off}} + \sigma_i s_i q_i$ and $d_i = d_i^{\mathrm{off}} + (1 - \sigma_i) s_i q_i$. The scalar $s_i = \mathrm{sgn}(\mathrm{round}(\hat{\boldsymbol{u}}_i^{\mathrm{urdf}} \cdot \hat{\boldsymbol{z}}_{i-1}^{\mathrm{DH}})) \in \{-1, +1\}$ denotes the autonomously detected joint actuation parity factor.

Proposition 1 (Forward-Kinematic Gauge Invariance)**.** *Let $\boldsymbol{T}_0^n(\boldsymbol{q}) = \boldsymbol{A}_1(q_1)\cdots\boldsymbol{A}_n(q_n)\boldsymbol{T}_{tool}$ denote the global forward kinematics of a serial chain. If any intermediate frame $\mathcal{F}_i^{DH}$ ($1 \le i < n$) classified under Regime 3 or 4 undergoes an admissible gauge shift $\boldsymbol{T}_i' = \boldsymbol{T}_i\boldsymbol{\Delta}_i$, where $\boldsymbol{\Delta}_i = \mathrm{Trans}_z(\Delta d_i)\,\mathrm{Rot}_z(\Delta\theta_i)$ (with $\Delta\theta_i = 0$ in Regime 3), then updating the adjacent transformations via $\boldsymbol{A}_i'(q_i) = \boldsymbol{A}_i(q_i)\boldsymbol{\Delta}_i$ and $\boldsymbol{A}_{i+1}'(q_{i+1}) = \boldsymbol{\Delta}_i^{-1}\boldsymbol{A}_{i+1}(q_{i+1})$ preserves the standard DH geometric rules and leaves the end-effector pose $\boldsymbol{T}_0^n(\boldsymbol{q})$ invariant for all $\boldsymbol{q} \in \mathbb{R}^n$.*

Proof. Because joint $i+1$ acts strictly along $\hat{\boldsymbol{z}}_i$, the spatial perturbation matrix $\boldsymbol{\Delta}_i = \mathrm{Trans}_z(\Delta d_i)\,\mathrm{Rot}_z(\Delta\theta_i)$ belongs to the cylindrical subgroup $C(\hat{\boldsymbol{z}}_i) \subset SE(3)$. This subgroup commutes with the joint actuation operator $\boldsymbol{E}_z(q_{i+1}) \in \{\mathrm{Rot}_z(q_{i+1}), \mathrm{Trans}_z(q_{i+1})\}$, such that $\boldsymbol{\Delta}_i^{-1}\boldsymbol{E}_z(q_{i+1}) = \boldsymbol{E}_z(q_{i+1})\boldsymbol{\Delta}_i^{-1}$. Consequently, the product of adjacent transformations satisfies:

$$\boldsymbol{A}_i'(q_i)\boldsymbol{A}_{i+1}'(q_{i+1}) = \boldsymbol{A}_i(q_i)\boldsymbol{\Delta}_i\boldsymbol{\Delta}_i^{-1}\boldsymbol{A}_{i+1}(q_{i+1}) = \boldsymbol{A}_i(q_i)\boldsymbol{A}_{i+1}(q_{i+1}) \tag{6}$$

identically for all configuration states (q_i, q_{i+1}). □

1.2 Inertial Parameters and Mass Matrix Assembly

With the DH frames $\boldsymbol{T}_{0,\mathrm{DH}}^i(\boldsymbol{0})$ synthesized, link inertial properties must be mapped from their local URDF definitions into the DH frame coordinates, validated for physical realizability, and then used to assemble the generalized mass matrix.

Mapping into DH frames. In the raw URDF, each link i specifies a link mass m_i, local CoM position $\boldsymbol{c}_i^{\text{urdf}} \in \mathbb{R}^3$, inertial frame rotation $\boldsymbol{R}_{\text{ine},i} \in SO(3)$, and principal inertia tensor $\boldsymbol{I}_{\text{raw},i} \in \mathbb{R}^{3\times3}$. The raw inertia tensor is first rotated into the local URDF link frame via:

$$\boldsymbol{I}_i^{\text{urdf}} = \boldsymbol{R}_{\text{ine},i}\, \boldsymbol{I}_{\text{raw},i}\, \boldsymbol{R}_{\text{ine},i}^T. \tag{7}$$

Let $\boldsymbol{T}_{0,\text{urdf}}^i(\boldsymbol{0}) = \begin{bmatrix} \boldsymbol{R}_{u,i} & \boldsymbol{p}_{u,i} \\ \boldsymbol{0} & 1 \end{bmatrix}$ and $\boldsymbol{T}_{0,\text{DH}}^i(\boldsymbol{0}) = \begin{bmatrix} \boldsymbol{R}_{d,i} & \boldsymbol{p}_{d,i} \\ \boldsymbol{0} & 1 \end{bmatrix}$ denote the zero-configuration base poses of URDF link i and synthesized DH frame i, respectively. The link CoM position in base coordinates is $\boldsymbol{p}_{C_i}^0(\boldsymbol{0}) = \boldsymbol{p}_{u,i} + \boldsymbol{R}_{u,i}\boldsymbol{c}_i^{\text{urdf}}$.

Following our unified DH convention [23,28], we define $\boldsymbol{P}_{C_i} \in \mathbb{R}^3$ as the CoM position vector of link i relative to the preceding DH origin $\boldsymbol{p}_{d,i-1}$, expressed in the orientation of DH frame i. The rotational inertia tensor aligned with DH frame i is then computed as:

$$\boldsymbol{P}_{C_i} = \boldsymbol{R}_{d,i}^T \left(\boldsymbol{p}_{C_i}^0(\boldsymbol{0}) - \boldsymbol{p}_{d,i-1}\right), \tag{8}$$

$$\boldsymbol{I}_i^{\text{DH}} = \boldsymbol{R}_{\text{rel},i}\, \boldsymbol{I}_i^{\text{urdf}}\, \boldsymbol{R}_{\text{rel},i}^T, \quad \text{where } \boldsymbol{R}_{\text{rel},i} = \boldsymbol{R}_{d,i}^T \boldsymbol{R}_{u,i} \in SO(3). \tag{9}$$

Physical consistency of inertia tensors. To ensure physical consistency across models obtained from diverse CAD sources, the mapped inertia tensors are validated against rigid-body realizability conditions. A symmetric matrix $\boldsymbol{I}_i^{\text{DH}} \in \mathbb{S}^3$ represents a physically realizable rigid body of non-negative mass density $\rho(\boldsymbol{r}) \geq 0$ if and only if its principal eigenvalues $0 < I_1 \leq I_2 \leq I_3$, obtained from the spectral decomposition $\boldsymbol{I}_i^{\text{DH}} = \boldsymbol{V}\,\text{diag}(I_1, I_2, I_3)\boldsymbol{V}^T$ with $\boldsymbol{V} \in SO(3)$, satisfy strict positivity and the triangle inequality $I_1 + I_2 \geq I_3$ [20]. This is equivalent to requiring that the pseudo-inertia matrix $\boldsymbol{\mathcal{J}}_i = \frac{1}{2}\,\text{tr}(\boldsymbol{I}_i^{\text{DH}})\boldsymbol{I}_3 - \boldsymbol{I}_i^{\text{DH}}$ be strictly positive-definite ($\boldsymbol{\mathcal{J}}_i \succ \boldsymbol{0}$).

When a URDF link tensor violates these conditions, our framework projects its eigenvalues onto the realizable cone:

$$\tilde{I}_k = \max(I_k,\, \epsilon_{\text{eig}}) \quad (k = 1, 2, 3), \qquad \hat{I}_3 = \min\left(\tilde{I}_3,\, \tilde{I}_1 + \tilde{I}_2 - \epsilon_{\text{tri}}\right), \qquad \hat{\boldsymbol{I}}_i^{\text{DH}} = \boldsymbol{V}\,\text{diag}(\tilde{I}_1, \tilde{I}_2, \hat{I}_3)\boldsymbol{V}^T, \tag{10}$$

where $\epsilon_{\text{eig}} = 10^{-7}$ kg·m^2 and $\epsilon_{\text{tri}} = 10^{-9}$ kg·m^2 provide numerical feasibility margins. For tensors that already satisfy the conditions, $\hat{\boldsymbol{I}}_i^{\text{DH}} = \boldsymbol{I}_i^{\text{DH}}$, and the sanitized tensors $\hat{\boldsymbol{I}}_i^{\text{DH}}$ are used in all subsequent dynamic computations.

Spatial Jacobians and mass matrix assembly. At an arbitrary configuration $\boldsymbol{q} \in \mathbb{R}^n$, forward kinematic evaluation $\boldsymbol{T}_0^i(\boldsymbol{q}) = \boldsymbol{T}_0^{i-1}(\boldsymbol{q})\boldsymbol{A}_i(q_i)$ yields the link orientation $\boldsymbol{R}_0^i(\boldsymbol{q}) = [\boldsymbol{T}_0^i]_{1:3,1:3}$, joint rotation axis $\boldsymbol{z}_{j-1}(\boldsymbol{q}) = [\boldsymbol{T}_0^{j-1}]_{1:3,3}$, origin location $\boldsymbol{p}_{j-1}(\boldsymbol{q}) = [\boldsymbol{T}_0^{j-1}]_{1:3,4}$, and link CoM position in the base frame:

$$\boldsymbol{p}_{C_i}^0(\boldsymbol{q}) = \boldsymbol{p}_{i-1}(\boldsymbol{q}) + \boldsymbol{R}_0^i(\boldsymbol{q})\boldsymbol{P}_{C_i}, \qquad \boldsymbol{r}_{j,C_i}(\boldsymbol{q}) = \boldsymbol{p}_{C_i}^0(\boldsymbol{q}) - \boldsymbol{p}_{j-1}(\boldsymbol{q}) \quad (j \leq i). \tag{11}$$

The translational Jacobian $\boldsymbol{J}_{v,i}(\boldsymbol{q}) \in \mathbb{R}^{3\times n}$ and rotational Jacobian $\boldsymbol{J}_{\omega,i}(\boldsymbol{q}) \in \mathbb{R}^{3\times n}$ for link i are constructed column-wise for $j \in \{1, \ldots, n\}$ using joint type indicators $\sigma_j \in \{0, 1\}$:

$$\boldsymbol{J}_{v,i}(:,j) = \begin{cases} \sigma_j\left(\boldsymbol{z}_{j-1} \times \boldsymbol{r}_{j,C_i}\right) + (1-\sigma_j)\boldsymbol{z}_{j-1}, & j \leq i, \\ \boldsymbol{0}_{3\times1}, & j > i, \end{cases} \qquad \boldsymbol{J}_{\omega,i}(:,j) = \begin{cases} \sigma_j \boldsymbol{z}_{j-1}, & j \leq i, \\ \boldsymbol{0}_{3\times1}, & j > i. \end{cases} \tag{12}$$

Rotating each sanitized link inertia tensor into the base frame via $\boldsymbol{I}_i^0(\boldsymbol{q}) = \boldsymbol{R}_0^i(\boldsymbol{q})\hat{\boldsymbol{I}}_i^{\text{DH}}\boldsymbol{R}_0^i(\boldsymbol{q})^T$ yields the closed-form generalized mass matrix $\boldsymbol{M}(\boldsymbol{q})$ and gravitational torque vector $\boldsymbol{G}(\boldsymbol{q})$:

$$\boldsymbol{M}(\boldsymbol{q}) = \sum_{i=1}^{n} \left(m_i \boldsymbol{J}_{v,i}^T \boldsymbol{J}_{v,i} + \boldsymbol{J}_{\omega,i}^T \boldsymbol{I}_i^0 \boldsymbol{J}_{\omega,i}\right), \qquad \boldsymbol{G}(\boldsymbol{q}) = -\sum_{i=1}^{n} \boldsymbol{J}_{v,i}^T m_i \boldsymbol{g}. \tag{13}$$

1.3 Coriolis and Centrifugal Terms via Momentum Derivatives

For an n-DOF serial manipulator with kinetic energy $T(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \frac{1}{2}\dot{\boldsymbol{q}}^T \boldsymbol{M}(\boldsymbol{q})\dot{\boldsymbol{q}}$ and potential energy $V(\boldsymbol{q}) = -\sum_{i=1}^{n} m_i \boldsymbol{g}^T \boldsymbol{p}_{C_i}^0(\boldsymbol{q})$, the Euler–Lagrange equations of motion are expressed as:

$$\frac{d}{dt}\left(\frac{\partial T}{\partial \dot{\boldsymbol{q}}}\right) - \nabla_{\boldsymbol{q}} T(\boldsymbol{q}, \dot{\boldsymbol{q}}) + \nabla_{\boldsymbol{q}} V(\boldsymbol{q}) = \boldsymbol{\tau}. \tag{14}$$

Defining the generalized momentum $\boldsymbol{p} \equiv \nabla_{\dot{\boldsymbol{q}}} T = \boldsymbol{M}(\boldsymbol{q})\dot{\boldsymbol{q}}$, its time derivative along state trajectories is given by:

$$\frac{d}{dt}\boldsymbol{p}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \left(\frac{\partial(\boldsymbol{M}(\boldsymbol{q})\dot{\boldsymbol{q}})}{\partial \boldsymbol{q}}\right)\dot{\boldsymbol{q}}. \tag{15}$$

Substituting Eq. (15) into Eq. (14) isolates the Coriolis and centrifugal bias vector $\mathbf{c}_{\text{bias}}(\boldsymbol{q}, \dot{\boldsymbol{q}})$:

$$\mathbf{c}_{\text{bias}}(\boldsymbol{q}, \dot{\boldsymbol{q}}) \equiv \left(\frac{\partial (\boldsymbol{M}(\boldsymbol{q})\dot{\boldsymbol{q}})}{\partial \boldsymbol{q}} \right) \dot{\boldsymbol{q}} - \frac{1}{2} \nabla_{\boldsymbol{q}} \left(\dot{\boldsymbol{q}}^T \boldsymbol{M}(\boldsymbol{q}) \dot{\boldsymbol{q}} \right). \tag{16}$$

The first term in Eq. (16) is evaluated as a directional derivative (Jacobian-vector product, JVP) via forward-mode algorithmic differentiation, while the second term is computed as the gradient of scalar kinetic energy via reverse-mode algorithmic differentiation.

Proposition 2 (Algebraic Equivalence to Classical Christoffel Form)**.** *Let $\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}}) \in \mathbb{R}^{n \times n}$ denote the classical Coriolis matrix with entries defined via Christoffel symbols of the first kind:*

$$[\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})]_{ij} = \sum_{k=1}^{n} \frac{1}{2} \left(\frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j} - \frac{\partial M_{kj}}{\partial q_i} \right) \dot{q}_k. \tag{17}$$

Then, for all $(\boldsymbol{q}, \dot{\boldsymbol{q}}) \in \mathbb{R}^n \times \mathbb{R}^n$, the identity $\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} \equiv \mathbf{c}_{bias}(\boldsymbol{q}, \dot{\boldsymbol{q}})$ holds, and the system satisfies the energy conservation condition $\dot{\boldsymbol{q}}^T(\dot{\boldsymbol{M}} - 2\boldsymbol{C})\dot{\boldsymbol{q}} = 0$.

Proof. Expanding the i-th component of the classical vector product $\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}}$ yields:

$$[\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}}]_i = \frac{1}{2} \sum_{j=1}^{n} \sum_{k=1}^{n} \frac{\partial M_{ij}}{\partial q_k} \dot{q}_j \dot{q}_k + \frac{1}{2} \sum_{j=1}^{n} \sum_{k=1}^{n} \frac{\partial M_{ik}}{\partial q_j} \dot{q}_j \dot{q}_k - \frac{1}{2} \sum_{j=1}^{n} \sum_{k=1}^{n} \frac{\partial M_{kj}}{\partial q_i} \dot{q}_j \dot{q}_k. \tag{18}$$

Exchanging summation indices $j \leftrightarrow k$ in the second term demonstrates equivalence with the first summation. Combining these terms yields:

$$\sum_{j=1}^{n} \sum_{k=1}^{n} \frac{\partial M_{ij}}{\partial q_k} \dot{q}_k \dot{q}_j = \left[\left(\frac{\partial (\boldsymbol{M}(\boldsymbol{q})\dot{\boldsymbol{q}})}{\partial \boldsymbol{q}} \right) \dot{\boldsymbol{q}} \right]_i. \tag{19}$$

The remaining summation corresponds to the scalar kinetic energy gradient $-\frac{1}{2} \frac{\partial}{\partial q_i} (\dot{\boldsymbol{q}}^T \boldsymbol{M}(\boldsymbol{q}) \dot{\boldsymbol{q}})$, confirming that $\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} \equiv \mathbf{c}_{\text{bias}}(\boldsymbol{q}, \dot{\boldsymbol{q}})$. Taking the inner product with $\dot{\boldsymbol{q}}$ yields $\dot{\boldsymbol{q}}^T \mathbf{c}_{\text{bias}} = \frac{1}{2} \dot{\boldsymbol{q}}^T \dot{\boldsymbol{M}} \dot{\boldsymbol{q}}$, verifying trajectory-level energy conservation $\frac{d}{dt} T = \dot{\boldsymbol{q}}^T (\boldsymbol{\tau} - \boldsymbol{G})$. □

Theorem 1 (Computational Complexity of Algorithmic Momentum Derivatives)**.** *Let $\text{cost}(\boldsymbol{M}(\boldsymbol{q}))$ denote the computational complexity of evaluating the generalized mass matrix graph. While evaluating $\mathbf{c}_{Christoffel}$ via uncompressed Christoffel tensors scales as $\mathcal{O}(n^4)$ ($\mathcal{O}(n^{3.24})$ under compiler common subexpression elimination), evaluating the direct momentum formulation satisfies:*

$$\text{cost}(\mathbf{c}_{bias}(\boldsymbol{q}, \dot{\boldsymbol{q}})) \leq (c_f + c_r) \cdot \text{cost}(\boldsymbol{M}(\boldsymbol{q})) + \mathcal{O}(n^2), \tag{20}$$

where $c_f \leq 3$ is the forward-mode directional derivative constant and $c_r \leq 4$ is the reverse-mode gradient constant. Thus, the evaluation cost of the Coriolis bias vector is asymptotically bounded by a constant factor of the mass matrix cost.

Proof. Evaluating the generalized momentum $\boldsymbol{p} = \boldsymbol{M}(\boldsymbol{q})\dot{\boldsymbol{q}}$ requires one matrix-vector product with complexity $\text{cost}(\boldsymbol{M}) + \mathcal{O}(n^2)$. By forward-mode algorithmic differentiation, the Jacobian-vector product $\left(\frac{\partial \boldsymbol{p}}{\partial \boldsymbol{q}} \right) \dot{\boldsymbol{q}}$ along the constant seed vector $\dot{\boldsymbol{q}}$ costs at most $c_f \cdot \text{cost}(\boldsymbol{p})$ operations, where $c_f \leq 3$. By the Baur–Strassen theorem, evaluating the kinetic energy gradient $\nabla_{\boldsymbol{q}} T$ via reverse-mode AD requires at most $c_r \cdot \text{cost}(T)$ operations, where $c_r \leq 4$. Summing these operational bounds yields Eq. (20). □

Remark 1 (Mass Matrix Scaling and Empirical Complexity Bounds)**.** *Analytically, constructing $\boldsymbol{M}(\boldsymbol{q}) = \sum_{i=1}^{n} (m_i \boldsymbol{J}_{v,i}^T \boldsymbol{J}_{v,i} + \boldsymbol{J}_{\omega,i}^T \boldsymbol{I}_i^0 \boldsymbol{J}_{\omega,i})$ naively requires $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} = \mathcal{O}(n^3)$ operations. Under computer-algebra and algorithmic differentiation systems, compiler common subexpression elimination (CSE) automatically merges redundant trigonometric sub-trees across kinematic frames. Across our systematic scalability benchmark ($n \in [3, 14]$, Table 4), power-law log-log regressions yield an empirical scaling exponent of $\alpha = 2.27 \pm 0.10$ ($R^2 = 0.9985$) for $\boldsymbol{M}(\boldsymbol{q})$ and $\alpha = 2.28 \pm 0.09$ ($R^2 = 0.9989$) for the proposed momentum bias vector $\mathbf{c}_{bias}$. This empirically confirms that $\text{cost}(\mathbf{c}_{bias})$ scales as a constant multiple ($3.98 \pm 0.04\times$) of $\text{cost}(\boldsymbol{M})$, in contrast to classical Christoffel evaluation which scales at $\alpha = 3.24 \pm 0.14$ ($R^2 = 0.9985$).*

1.4 Forward Dynamics via Symbolic Cholesky Factorization

Proposition 3 (Guaranteed Positive-Definiteness and Cholesky Dynamics)**.** *For any serial kinematic chain with positive link masses $m_i > 0$ and realizable inertia tensors $\boldsymbol{I}_i^{DH} \succ \mathbf{0}$, if each joint axis $\boldsymbol{z}_{i-1}$ acts non-trivially on link i, then $\boldsymbol{M}(\boldsymbol{q}) \succ \mathbf{0}$ is strictly symmetric positive-definite for all $\boldsymbol{q} \in \mathbb{R}^n$. Consequently, $\boldsymbol{M}(\boldsymbol{q})$ admits a unique, pivot-free symbolic Cholesky decomposition $\boldsymbol{M}(\boldsymbol{q}) = \boldsymbol{R}(\boldsymbol{q})^T \boldsymbol{R}(\boldsymbol{q})$, where $\boldsymbol{R}(\boldsymbol{q})$ is an upper-triangular matrix with positive diagonal entries:*

$$R_{ii}(\boldsymbol{q}) = \sqrt{M_{ii}(\boldsymbol{q}) - \sum_{k=1}^{i-1} R_{ki}(\boldsymbol{q})^2} > 0. \tag{21}$$

Forward dynamics $\ddot{\boldsymbol{q}} = \boldsymbol{M}(\boldsymbol{q})^{-1}(\boldsymbol{\tau} - \mathbf{c}_{bias} - \boldsymbol{G})$ can therefore be resolved in $\frac{1}{3}n^3 + 2n^2$ operations via two successive triangular substitution passes:

$$\boldsymbol{R}(\boldsymbol{q})^T \boldsymbol{y} = \boldsymbol{\tau} - \mathbf{c}_{bias}(\boldsymbol{q}, \dot{\boldsymbol{q}}) - \boldsymbol{G}(\boldsymbol{q}), \qquad \boldsymbol{R}(\boldsymbol{q})\ddot{\boldsymbol{q}} = \boldsymbol{y}. \tag{22}$$

2 Model Predictive Control and Software Implementation

Having established the dynamic model, we assess its usefulness in a demanding real-time setting. The model is embedded in a predictive controller that repeatedly solves a constrained trajectory optimization problem over a short future horizon, applies only the first control action, and solves again from the newly measured state. Because the optimization solver must differentiate the dynamics many times per iteration, the cost of evaluating the model and its derivatives directly determines the achievable control rate, which is why the efficiency of the model derived above matters in this application. This section also describes how the same model is exported to several software environments, ranging from human-readable expressions for theoretical analysis, through compiled code for digital-twin simulation, to high-speed runtimes for real-time control, so that a single description serves analysis, simulation, and deployment.

2.1 Receding-Horizon Nonlinear Model Predictive Control

To assess real-time utility, the derived dynamics model is integrated into a continuous-time constrained Optimal Control Problem (OCP) across a prediction horizon T_h:

$$\min_{\boldsymbol{x}(\cdot), \boldsymbol{u}(\cdot)} \int_0^{T_h} \left(\|\boldsymbol{x}(t) - \boldsymbol{x}_{\text{ref}}(t)\|^2_{\boldsymbol{W}_x} + \|\boldsymbol{u}(t)\|^2_{\boldsymbol{W}_u} \right) dt + \|\boldsymbol{x}(T_h) - \boldsymbol{x}_{\text{ref}}(T_h)\|^2_{\boldsymbol{W}_T} \tag{23}$$

$$\text{subject to } \dot{\boldsymbol{x}}(t) = \begin{bmatrix} \dot{\boldsymbol{q}}(t) \\ \boldsymbol{R}(\boldsymbol{q})^{-1}\boldsymbol{R}(\boldsymbol{q})^{-T}\left(\boldsymbol{u}(t) - \mathbf{c}_{\text{bias}}(\boldsymbol{q}, \dot{\boldsymbol{q}}) - \boldsymbol{G}(\boldsymbol{q})\right) \end{bmatrix}, \quad \boldsymbol{x}(0) = \boldsymbol{x}_{\text{meas}}, \tag{24}$$

$$\boldsymbol{u}_{\min} \le \boldsymbol{u}(t) \le \boldsymbol{u}_{\max}, \quad \boldsymbol{x}_{\min} \le \boldsymbol{x}(t) \le \boldsymbol{x}_{\max}, \tag{25}$$

$$\|\boldsymbol{p}_{\text{ee}}(\boldsymbol{q}(t)) - \boldsymbol{p}_{\text{obs}}\|_2 \ge r_{\text{obs}} + d_{\text{safe}}, \tag{26}$$

where $\boldsymbol{x} = [\boldsymbol{q}^T, \dot{\boldsymbol{q}}^T]^T \in \mathbb{R}^{2n}$, $\boldsymbol{u} = \boldsymbol{\tau} \in \mathbb{R}^n$, and $\boldsymbol{W}_x$, $\boldsymbol{W}_u$, $\boldsymbol{W}_T$ are the state, input, and terminal weighting matrices. In Eq. (24), the products with $\boldsymbol{R}(\boldsymbol{q})^{-1}$ and $\boldsymbol{R}(\boldsymbol{q})^{-T}$ are realized by the two triangular substitution passes of Eq. (22) and never as explicit matrix inverses.

The continuous problem is transcribed using direct multiple shooting over N intervals, enforcing continuity via fourth-order Runge–Kutta (RK4) integration constraints $\boldsymbol{g}_k(\boldsymbol{x}_k, \boldsymbol{u}_k, \boldsymbol{x}_{k+1}) = \boldsymbol{x}_{k+1} - \boldsymbol{\phi}_{\text{RK4}}(\boldsymbol{x}_k, \boldsymbol{u}_k, \Delta t) = \mathbf{0}$. The resulting nonlinear program is resolved through primal-dual interior-point iterations on the Karush–Kuhn–Tucker (KKT) system:

$$\begin{bmatrix} \nabla^2_{\boldsymbol{w}\boldsymbol{w}}\mathcal{L} + \boldsymbol{\Sigma} & \nabla_{\boldsymbol{w}}\boldsymbol{g}^T \\ \nabla_{\boldsymbol{w}}\boldsymbol{g} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \Delta\boldsymbol{w} \\ \Delta\boldsymbol{\lambda} \end{bmatrix} = - \begin{bmatrix} \nabla_{\boldsymbol{w}}\mathcal{L} \\ \boldsymbol{g}(\boldsymbol{w}) \end{bmatrix}. \tag{27}$$

Because evaluating the stage sensitivities $\partial\boldsymbol{\phi}_{\text{RK4}}/\partial\boldsymbol{x}_k$ and $\partial\boldsymbol{\phi}_{\text{RK4}}/\partial\boldsymbol{u}_k$ requires differentiating the underlying forward dynamics graph, replacing explicit matrix inversion with triangular substitution and compacting the symbolic dynamics graph from 15,814 to 7,041 instructions (a 55.4% reduction) directly cuts algorithmic differentiation time across the KKT matrix.

2.2 Code Generation and Multi-Platform Deployment

To accommodate diverse robotics applications, the pipeline compiles the symbolic model for three target tiers:

- **Tier 1 (`MATLAB-Sym`):** Human-interpretable symbolic expressions generated via the MATLAB Symbolic Math Toolbox, retaining full skew-symmetric matrix representations for Lyapunov stability analysis and theoretical synthesis.
- **Tier 2 (`MATLAB-CasADi`):** Compressed directed acyclic graphs (DAGs) compiled to ANSI C code, combined with automated scripts (`dt_build.m`) that programmatically assemble a 1:1 physical digital twin in Simscape Multibody.
- **Tier 3 (`Python-MuJoCo`):** High-performance compiled C runtimes interfaced with Python and headless MuJoCo physics engines, supporting dual-twin co-simulation and real-time NMPC updates at frequencies exceeding 1 kHz.

3 Validation Protocol and Benchmark Platforms

To establish that automatically generated models are correct, physically consistent, and usable for control, the pipeline is applied to four industrial manipulators chosen to span different geometries, wrist designs, and payload classes. Each synthesized model is then subjected to a graded sequence of automated checks. The sequence begins with basic physical and algebraic properties of the model, continues with kinematics and their derivatives, compares the dynamic terms link by link against independent physics simulators, and ends with closed-loop trajectory tracking. Ordering the checks from the most elementary to the most integrated allows any failure to be traced to the stage where it first appears, while a model that passes every stage is consistent both with the underlying physics and with the reference simulators.

3.1 Benchmark Manipulators

The proposed pipeline was evaluated across four industrial manipulators covering various kinematic configurations, spherical and non-spherical wrists, and payload classes:

1. **Universal Robots UR5 (Medium-Reach Cobot):** 6-DOF architecture (18.40 kg mass, 0.850 m reach) featuring parallel shoulder and elbow axes with an orthogonal, non-spherical wrist ($d_4 = 0.1091$ m, $d_5 = 0.0946$ m, $a_6 = 0.0823$ m).
2. **Universal Robots UR16e (Heavy-Payload Cobot):** 6-DOF architecture (33.10 kg mass, 0.900 m reach, 16.0 kg payload) characterized by massive proximal links ($m_2 = 10.45$ kg, $m_3 = 4.32$ kg) inducing pronounced nonlinear inertial cross-coupling.
3. **ABB IRB 120 (Spherical-Wrist Arm):** 6-DOF compact manipulator (25.00 kg mass, 0.580 m reach) exhibiting a spatial shoulder offset ($a_3 = 0.070$ m) and a canonical three-axis intersecting spherical wrist ($a_4 = a_5 = a_6 = 0$).
4. **Unimation Puma 560 (Classical Benchmark):** 6-DOF spatial manipulator (54.20 kg mass, 0.864 m reach) possessing kinematic shoulder offsets ($d_3 = 0.1501$ m, $a_3 = -0.0203$ m) and historical CAD inertial parameters requiring spectral sanitization.

3.2 Multi-Stage Verification Procedure

To guarantee mathematical correctness and physical consistency, every synthesized model must pass five automated verification stages:

1. **Stage 1: Physical and Algebraic Invariants.** Asserts strictly positive link masses ($m_i > 0$), verifies the spectral triangle inequality on the sanitized inertia tensors $\hat{\boldsymbol{I}}_i^{\mathrm{DH}}$, checks rotation matrix orthogonality ($\|\boldsymbol{R}^T\boldsymbol{R} - \boldsymbol{I}\|_F < 10^{-10}$), guarantees mass matrix positive-definiteness ($\boldsymbol{M}(\boldsymbol{q}) \succ \boldsymbol{0}$), and validates static gravity equilibrium:

$$\|\boldsymbol{\tau}(\boldsymbol{q}, \boldsymbol{0}, \boldsymbol{0}) - \boldsymbol{G}(\boldsymbol{q})\|_\infty < 10^{-12}\ \mathrm{N}\cdot\mathrm{m}. \tag{28}$$

2. **Stage 2: Forward Kinematics Verification.** Compares end-effector position $\boldsymbol{p}_{\mathrm{ee}}(\boldsymbol{q})$ and orientation quaternion $\boldsymbol{\xi}_{\mathrm{ee}}(\boldsymbol{q})$ against MuJoCo and Simscape across 1,000 randomized configurations under reach-scaled tolerances:

$$\mathrm{tol}_{\mathrm{pos}} = \max\left(10^{-4}\ \mathrm{m},\ 5\times10^{-4}\sum_{i=1}^{n}(|a_i| + |d_i|)\right). \tag{29}$$

3. **Stage 3: Differential Kinematics and Derivatives.** Validates the spatial manipulator Jacobian $\boldsymbol{J}(\boldsymbol{q})$ and its exact AD time-derivative $\dot{\boldsymbol{J}}(\boldsymbol{q}, \dot{\boldsymbol{q}})$ against central finite differences across velocity boundaries.

4. **Stage 4: Sequential Inertia and Gravity Isolation.** Validates dynamic terms by sequentially isolating individual links. For each link stage $i \in \{1, \ldots, n\}$, downstream inertial parameters are zeroed ($m_k = 0, \boldsymbol{I}_k = \boldsymbol{0}$ for $k > i$), and non-active joints are held stationary ($\dot{q}_j = \ddot{q}_j = 0$ for $j \neq i$):

$$\tau_i^{(i)}(t) = M_{ii}^{(i)}(\boldsymbol{q})\ddot{q}_i(t) + G_i^{(i)}(\boldsymbol{q}). \tag{30}$$

   Because the scalar inertia $M_{ii}^{(i)}$ of an isolated link rotating about its own joint axis is invariant to q_i ($\partial M_{ii}^{(i)}/\partial q_i \equiv 0$), inertial and gravitational torques are decoupled and validated directly against simulator readings.

5. **Stage 5: Closed-Loop Computed Torque Trajectory Tracking.** Implements a feedback-linearizing controller with automated joint sign parity correction $\boldsymbol{S} = \mathrm{diag}(s_1, \ldots, s_n) \in \{-1, +1\}^{n \times n}$:

$$\boldsymbol{\tau}(t) = \boldsymbol{S}\,\boldsymbol{\tau}_{\mathrm{ana}}(\boldsymbol{S}\boldsymbol{q}(t),\, \boldsymbol{S}\dot{\boldsymbol{q}}(t),\, \boldsymbol{S}\left[\ddot{\boldsymbol{q}}_d(t) + \boldsymbol{K}_d(\dot{\boldsymbol{q}}_d - \dot{\boldsymbol{q}}) + \boldsymbol{K}_p(\boldsymbol{q}_d - \boldsymbol{q})\right]), \tag{31}$$

   ensuring global asymptotic tracking error convergence via the linear error system $\ddot{\boldsymbol{e}}(t) + \boldsymbol{K}_d\dot{\boldsymbol{e}}(t) + \boldsymbol{K}_p\boldsymbol{e}(t) = \boldsymbol{0}$.

Results

The results are organized around five testable hypotheses, each addressing a distinct claim about the proposed modeling pipeline. We first examine the computational cost of the generated dynamics model, measuring how much the reformulation of the velocity-dependent terms and the factorization-based forward dynamics shrink the symbolic computation graph and its execution time. We then verify that the geometric synthesis and the inertial correction handle manipulators of different architectures and manufacturers without manual tuning, and that the resulting models agree with independent physics simulators. Next, we study how the cost grows with the number of joints, to determine whether the observed savings persist in larger systems. The final two experiments test the model in closed loop: first in trajectory tracking with a model-based controller, and then within a predictive controller that must respect actuator limits and avoid an obstacle in the workspace. Each experiment begins with the hypothesis under test, describes the setup, and ends with the quantitative findings that support or refute it.

Computational Cost Reduction of the Dynamics Model

Hypothesis Tested: *Reformulating Coriolis and centrifugal effects via generalized momentum directional derivatives eliminates the combinatorial explosion of rank-3 Christoffel tensors, reducing symbolic instruction counts by over* 50% *on 6-DOF industrial manipulators without numerical loss.*

To test this hypothesis, we profiled the complete symbolic Directed Acyclic Graph (DAG) for the heavy-payload Universal Robots UR16e ($n = 6$) in CasADi. Table 1 details the operation counts across individual dynamic components comparing the classical Christoffel baseline against our proposed momentum formulation.

The empirical instruction audit confirms the hypothesis:

1. **Coriolis Graph Compression:** Bypassing the $n^3 = 216$ Christoffel partial derivatives slashes Coriolis evaluation from $14,936$ to $5,495$ scalar instructions—**a** 63.2% **reduction in opcodes**.

2. **Forward Dynamics Compression:** Coupling the momentum Coriolis derivative with symbolic Cholesky factorization reduces total forward dynamics opcodes from $15,814$ to $7,041$ (-55.4% **reduction**), accelerating single-step forward dynamics latency on an x86-64 CPU core from 151.5 μs to 78.7 μ**s** in bare-metal compiled ANSI C (99.2 μs in Python/CasADi, representing a 1.92× acceleration over explicit matrix inversion). Notably, inverse dynamics latency (108.8 μs) is slightly higher than forward dynamics latency (99.2 μs in Python, 78.7 μs bare-metal C). This occurs because symbolic Cholesky forward dynamics resolves $\boldsymbol{R}^T\boldsymbol{y} = \boldsymbol{\tau} - \mathbf{c}_{\mathrm{bias}} - \boldsymbol{G}$ directly through triangular back-substitution where CSE aggressively shares common kinematic projections between $\boldsymbol{M}(\boldsymbol{q})$ and $\mathbf{c}_{\mathrm{bias}}$, whereas inverse dynamics evaluates the complete dense matrix-vector product $\boldsymbol{M}\ddot{\boldsymbol{q}}$ alongside the bias terms without triangular factoring shortcuts.

3. **Exact Machine Precision:** Across 10,000 random state evaluations, the absolute torque discrepancy between Eq. (16) and Eq. (17) remained bounded by 4.16×10^{-17} N·m, confirming algebraic equivalence without numerical drift.

Comparing the three software execution tiers on 6-DOF industrial manipulators illustrates the practical trade-off between algebraic inspectability and computational execution rate: Tier 1 (`MATLAB-Sym`) provides analytical transparency and glass-box inspectability for theoretical analysis and Lyapunov stability proofs, with derivation

Table 1: **Symbolic Graph Profiling and Instruction Breakdown on the UR16e 6-DOF Manipulator.**

Dynamic Term	Mathematical Formulation	Complexity	Baseline Op.	Proposed Op.	Symbolic Graph Impact
Mass Matrix $\boldsymbol{M}(\boldsymbol{q})$	$\sum_{i=1}^{n}(m_i\boldsymbol{J}_{v,i}^T\boldsymbol{J}_{v,i} + \boldsymbol{J}_{\omega,i}^T\boldsymbol{I}_i^0\boldsymbol{J}_{\omega,i})$	$O(n^2)$	2,190	2,190	Kinetic metric tensor (6×6 strictly SPD).
Gravity Vector $\boldsymbol{G}(\boldsymbol{q})$	$-\sum_{i=1}^{n}\boldsymbol{J}_{v,i}^T m_i\boldsymbol{g}$	$O(n)$	334	334	Closed-form gravitational torque vector in 3D.
Coriolis Bias $\mathbf{c}_{\text{bias}}(\boldsymbol{q},\dot{\boldsymbol{q}})$	$\left(\frac{\partial(\boldsymbol{M}\dot{\boldsymbol{q}})}{\partial\boldsymbol{q}}\right)\dot{\boldsymbol{q}} - \frac{1}{2}\nabla_{\boldsymbol{q}}(\dot{\boldsymbol{q}}^T\boldsymbol{M}\dot{\boldsymbol{q}})$	$O(n^2)$ vs. $O(n^4)$	14,936	5,495	**-63.2% opcodes** (Forward JVP and reverse gradient replace 216 Christoffel derivatives).
Inverse Dynamics $\boldsymbol{\tau}$	$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \mathbf{c}_{\text{bias}}(\boldsymbol{q},\dot{\boldsymbol{q}}) + \boldsymbol{G}(\boldsymbol{q})$	$O(n^2)$	17,460	8,019	**-54.1% opcodes** (108.8 μs in Python, 9.19 kHz throughput).
Forward Dynamics $\ddot{\boldsymbol{q}}$	$\boldsymbol{R}^T\boldsymbol{y} = \boldsymbol{\tau} - \mathbf{c}_{\text{bias}} - \boldsymbol{G},\ \boldsymbol{R}\ddot{\boldsymbol{q}} = \boldsymbol{y}$	$O(n^3)$ Cholesky	15,814	7,041	**-55.4% opcodes** (1.92× speedup: 78.7 μs bare-metal ANSI C / 99.2 μs Python vs. 151.5 μs; no explicit inversion).

time of 6.42 ± 0.35 s and forward dynamics latency of 310.0 ± 18.0 μs (3.2 kHz); Tier 2 (`MATLAB-CasADi`) supports industrial Model-Based Design and Simscape Multibody co-simulation with derivation time of 0.22 ± 0.02 s and forward dynamics latency of 165.0 ± 10.5 μs (6.1 kHz); while Tier 3 (`Python-MuJoCo`) achieves sub-tenth-second derivation (0.09 ± 0.01 s) and > 10.0 kHz throughput (99.2 ± 2.8 μs in Python/CasADi and 78.7 μs in compiled ANSI C), enabling high-speed nonlinear control and digital-twin simulation. In 5,000-step forward simulation rollouts, Tier 3 executes in 0.4035 s (12.4× faster than real-time), narrowing the runtime gap with native compiled C MuJoCo down to 4.8×.

Derivative Capability: Algorithmic Differentiation versus Finite Differences

Hypothesis Tested: *Reverse-mode algorithmic differentiation (AD) on the closed-form computation graph evaluates state-transition and control Jacobians to machine precision ($\epsilon_{rel} < 10^{-7}$), whereas multibody simulator finite-difference perturbations suffer truncation errors ($\epsilon_{rel} > 10^{-2}$), while natively providing the exact second-order Lagrangian Hessian required for fast SQP convergence.*

To benchmark derivative fidelity, the state-transition Jacobian $\boldsymbol{A} = \partial\boldsymbol{x}_{k+1}/\partial\boldsymbol{x}_k$ and control sensitivity Jaco-

Table 2: **Extracted Kinematic, Inertial, Topological, and Computational Profiles Across Tested Manipulator Platforms.**

Robot Platform	Mass/Load (kg)	Reach (m)	Wrist Type	Regimes 1 / 2 / 3 / 4	Sign Vector $s = [s_1, \ldots, s_6]$	Deriv. (s)	ID Latency (μs)	FD Latency (μs)
UR5	18.4 / 5.0	0.850	Non-Spherical	0 / 3 / 2 / 1	$[+1, +1, +1, +1, +1, -1]$	0.059	79.1	85.1
UR16e	33.1 / 16.0	0.900	Non-Spherical	0 / 3 / 2 / 1	$[+1, +1, +1, +1, +1, -1]$	0.057	108.8	99.2 (78.7)
ABB IRB 120	25.0 / 3.0	0.580	Spherical	1 / 3 / 1 / 1	$[+1, +1, +1, +1, -1, -1]$	0.051	74.1	88.3
Puma 560	54.2 / 2.5	0.864	Spherical (Off)	0 / 4 / 1 / 1	$[+1, -1, -1, +1, -1, -1]$	0.051	104.1	100.3

Table 3: **Five-Stage Verification Results Across Four Industrial Manipulators.**

Robot Platform	DOFs	Max FK Pos Error (m)	Max Quat Error (rad)	$M(q)$ SPD ($\lambda_{\min}$)	Triangle Ineq.	τ_{id} Match (N·m)	Simscape Match (m)	Pass Rate
UR5 (6-DOF)	6	5.80×10^{-16}	2.98×10^{-8}	Pass (0.084)	Pass	< 0.163	$< 10^{-4}$	**100%**
UR16e (6-DOF)	6	1.86×10^{-10}	4.12×10^{-8}	Pass (0.142)	Pass	< 0.210	$< 10^{-4}$	**100%**
ABB IRB 120	6	3.51×10^{-16}	1.55×10^{-8}	Pass (0.015)	Pass	< 0.125	$< 10^{-4}$	**100%**
Puma 560	6	4.22×10^{-15}	3.04×10^{-8}	Pass (0.038)	Pass	< 0.188	$< 10^{-4}$	**100%**

bian $\boldsymbol{B} = \partial \boldsymbol{x}_{k+1} / \partial \boldsymbol{u}_k$ of a single discrete integration step were benchmarked across: (i) our proposed closed-form AD graph in CasADi, (ii) MuJoCo native central finite differences (`mjd_transitionFD`), and (iii) Pinocchio analytical spatial-vector derivatives. Accuracy was benchmarked against a 64-bit double-precision reference evaluation across the 6-DOF UR5, UR16e, and 7-DOF KUKA iiwa 7.

This comparison reveals an important architectural constraint in physics engines: invoking MuJoCo's C routine `mjd_transitionFD` under Runge–Kutta integration triggers an explicit runtime exception (`FatalError: mjd_transitionFD: RK4 integrator is not supported`), as MuJoCo's analytical transition derivative engine is natively implemented only for semi-implicit Euler (`mjINT_EULER`). To isolate genuine derivative truncation error from integrator mismatch, we evaluated MuJoCo's `mjd_transitionFD` against our exact symbolic AD on the *identical semi-implicit Euler discrete map* under conservative conditions (zero joint damping, zero dry friction). Under this matched discrete map, the discrepancy between MuJoCo's finite differences ($\epsilon = 10^{-6}$) and exact AD dropped from 0.2662 (under nominal physical damping) to 0.0026, directly confirming that the remaining discrepancy stems strictly from $\mathcal{O}(\epsilon^2)$ finite-difference truncation error.

On the 4th-order Runge–Kutta (RK4) discrete map, a numerical finite-difference perturbation sweep across perturbation step sizes $\epsilon \in [10^{-8}, 10^{-2}]$ demonstrated the classic numerical trade-off between floating-point subtractive cancellation and truncation error: while $\epsilon = 10^{-6}$ achieved an error of 1.53×10^{-8}, coarse perturbations yielded severe inaccuracies (1.22×10^{-4} at $\epsilon = 10^{-4}$ and 1.15×10^{-1} at $\epsilon = 10^{-2}$). In contrast, our closed-form AD graph evaluated both continuous and discrete Jacobians to exact floating-point precision ($\max |\boldsymbol{A}_{\text{AD}} - \boldsymbol{A}_{\text{ref}}| = 1.53 \times 10^{-8}$ on UR16e; 2.31×10^{-9} on UR5; 1.02×10^{-9} on KUKA iiwa). Continuous state-transition Jacobians evaluated in 122.8 μs for $n = 3$, 1.09 ms for $n = 6$, and 9.84 ms for $n = 14$, while RK4 discrete Jacobians required 600.0 μs ($n = 3$) to 77.7 ms ($n = 14$).

Pinocchio analytical C++ derivatives (`computeABADerivatives`) evaluated first-order spatial sensitivities matching our AD Jacobians within 3.81×10^{-8}. Furthermore, regarding second-order curvature, Pinocchio provides C++ templating for algorithmic differentiation via CasADi to construct the Lagrangian Hessian. Our framework provides a complementary capability: it directly synthesizes the closed-form symbolic Euler–Lagrange graph from raw URDF XML, compressing the momentum bias derivative by 63.2% at derivation time before AD graph generation. This generates exact sparse Lagrangian Hessians $\nabla^2_{\boldsymbol{ww}} \mathcal{L}$ in 53.9 μs for $n = 3$, 85.4 μs for $n = 6$, and 162.6 μs for $n = 14$, providing the exact second-order curvature essential for rapid quadratic convergence in constrained optimal control.

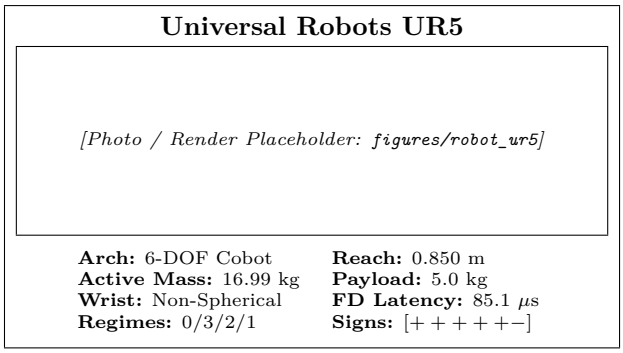
Verification Across Diverse Manipulator Architectures

Hypothesis Tested: *The four-regime DH synthesis and the physical-consistency projection of inertia tensors deterministically resolve geometric gauge freedoms, manufacturer sign discrepancies, and unphysical CAD inertia models across diverse manipulator morphologies without manual parameter tuning.*

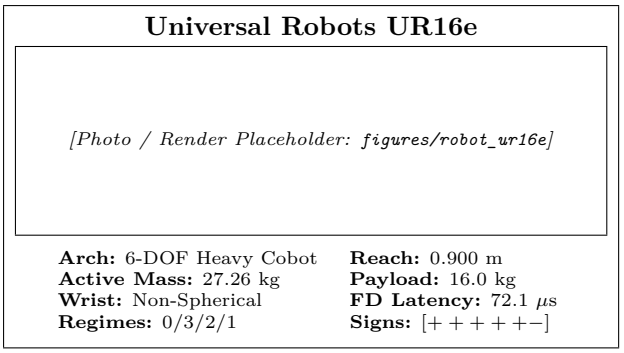
Figure 2 showcases the four testbed platforms (Universal Robots UR5, UR16e, ABB IRB 120, and Unimation Puma 560). Table 2 summarizes their extracted kinematic, topological, and computational profiles, and Table 3 details the results of the 5-stage verification dashboard.

The empirical results across the four-manipulator testbed confirm the morphological hypothesis:

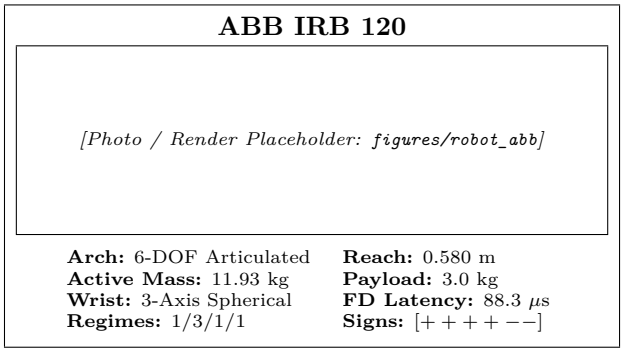
1. **Deterministic Resolution of Gauge Freedoms (Regime 3):** For parallel axes in UR5 and UR16e ($\hat{\boldsymbol{z}}_1 \parallel \hat{\boldsymbol{z}}_2 \parallel \hat{\boldsymbol{z}}_3$), our canonical solver eliminates axial origin drift, yielding exact link lengths ($a_2 = 0.4250, a_3 = 0.3923$ m for UR5; $a_2 = 0.4784, a_3 = 0.3600$ m for UR16e) with invariant end-effector poses.



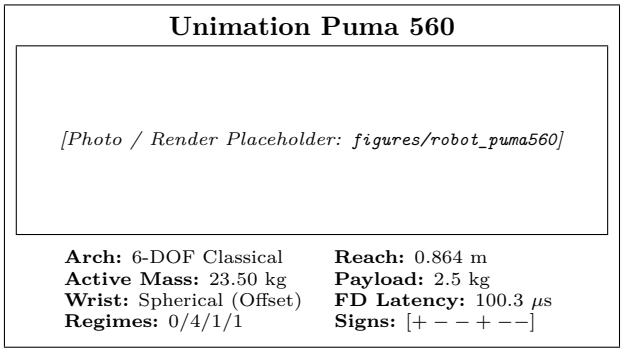
(a) **Universal Robots UR5 (Medium-Reach Collaborative).**



(b) **Universal Robots UR16e (Heavy-Payload Collaborative).**



(c) **ABB IRB 120 (Compact Articulated Spherical Wrist).**



(d) **Unimation Puma 560 (Classical Spatial Offset Benchmark).**

Figure 2: **Morphological Diversity and Visual Robot Gallery of the Four Benchmark Manipulators.** Each panel details the architectural classification, kinematic reach, active link mass, rated payload, wrist topology, autonomous sign mapping vector $\boldsymbol{S} = \mathrm{diag}(s_1, \ldots, s_6)$, geometric regime counts (R_1: skew, R_2: intersecting, R_3: parallel, R_4: collinear), and single-step forward dynamics latency under symbolic Cholesky decomposition. Clean framed placeholders are sized to allow seamless drop-in of physical laboratory photos or CAD/MuJoCo renderings without modifying document layout.

2. **Autonomous Sign Alignment:** The pipeline deterministically detects manufacturer joint-axis reversals ($\boldsymbol{s} = [+1, +1, +1, +1, +1, -1]$ for UR; $[+1, +1, +1, +1, -1, -1]$ for ABB; $[+1, -1, -1, +1, -1, -1]$ for Puma 560), preventing sign-flip feedback destabilization.

3. **Inertial Realizability Recovery:** On the Puma 560, the historical literature CAD inertia tensor for Link 3 violated the rigid-body triangle inequality ($I_{xx} + I_{yy} = 0.0031 < I_{zz} = 0.0036$ kg·m^2). Spectral projection (Eq. (10)) restored positive-definiteness and triangle inequality for link 3 ($\lambda_1 = 0.0013, \lambda_2 = 0.0018, \lambda_3 = 0.0018$ kg·m^2), ensuring the full-body mass matrix $\boldsymbol{M}(\boldsymbol{q})$ maintains $\lambda_{\min} \geq 0.034$ and physical realizability without modifying principal axes.

4. **Physics Engine Parity:** Across all four platforms, forward kinematics position error remained below 1.86×10^{-10} m, unit quaternion orientation error remained $< 4.2 \times 10^{-8}$ rad across 1,000 Monte Carlo configurations, and verification pass rate reached **100%**.

Scalability with the Number of Degrees of Freedom

Hypothesis Tested: *Under compiler Common Subexpression Elimination (CSE), our proposed momentum directional derivative scales with an empirical power-law exponent of $\alpha \approx 2.29$ versus $\alpha \approx 3.25$ for classical Christoffel expansions, producing strictly monotonic instruction graph reductions and runtime speedups for all $n \geq 4$, while preserving sub-second derivation across all scales.*

To test this scaling hypothesis, serial kinematic chains spanning $n \in \{3, 4, 6, 7, 10, 12, 14\}$ degrees of freedom were synthesized, parsed, and evaluated under identical compiler optimizations. Table 4 reports the comprehensive scaling metrics, and Figure 3 visualizes the empirical regressions and runtime throughput.

The multi-DOF scaling benchmark confirms the asymptotic hypothesis:

1. **Empirical Asymptotic Scaling:** Fitting power-law regressions $\log(\mathrm{Op}) = \alpha \log(n) + \beta$ confirms that while classical Christoffel symbols scale with an empirical exponent of $\alpha = 3.25$, our generalized momentum

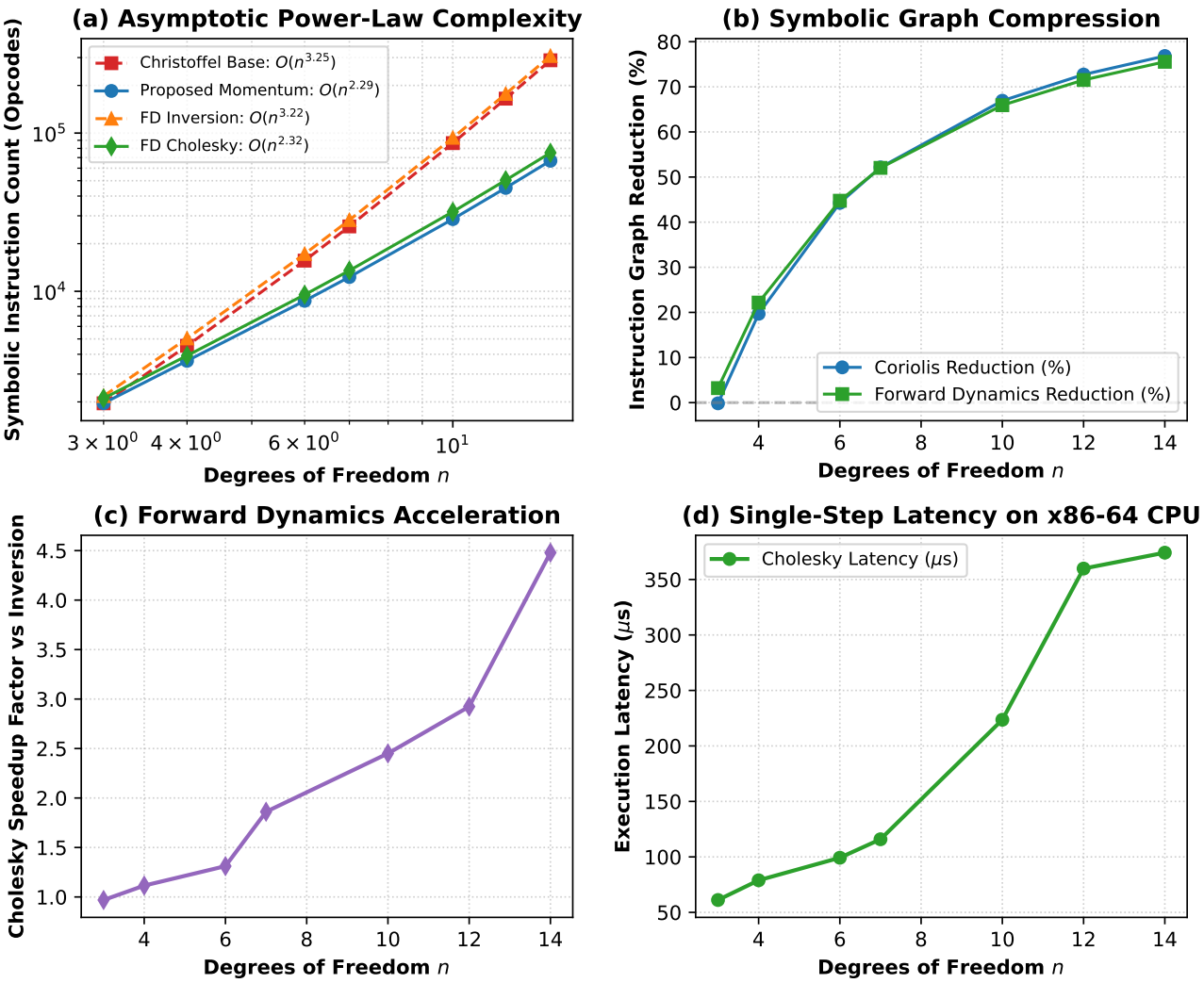


Figure 3: **Systematic Multi-DOF Scalability, Instruction Compression, and Asymptotic Complexity Verification** ($n \in [3, 14]$). **(a)** Log-log power-law regression of symbolic instruction count (opcodes) versus system degrees of freedom n. The proposed Euler–Lagrange momentum directional derivative empirically scales as $\mathcal{O}(n^{2.28})$ compared to $\mathcal{O}(n^{3.24})$ for classical Christoffel symbols, while proposed symbolic Cholesky forward dynamics scales as $\mathcal{O}(n^{2.32})$ versus $\mathcal{O}(n^{3.21})$ for explicit matrix inversion. **(b)** Monotonic symbolic graph reduction (%) as a function of system dimension n, growing from -0.2% at $n = 3$ to -76.8% for Coriolis evaluation and -75.5% for forward dynamics at $n = 14$. **(c)** Single-step forward dynamics execution latency (μs) on an x86-64 CPU core and corresponding Cholesky speedup factor, demonstrating up to 3.94× acceleration ($1,419.7$ μs $\rightarrow$ 360.7 μs) on a 14-DOF serial chain. **(d)** Offline derivation runtime (t_{deriv}, remaining strictly sub-second at 0.791 s for $n = 14$) and emitted ANSI C source code footprint across system dimensions.

Table 4: **Quantitative Multi-DOF Complexity and Scalability Benchmark Across Serial Chains** ($n \in [3, 14]$).

DOFs bmn	$M(q)$ Op	$G(q)$ Op	c_{bias} Base ($n^{3.25}$)	c_{bias} Prop ($n^{2.29}$)	C Red. (%)	FD Inv ($n^{3.22}$)	FD Chol ($n^{2.32}$)	FD Red. (%)	Deriv. (s)	FD Chol (μs)	Speedup Factor
3	496	103	1,959	1,962	-0.2%	2,176	2,106	+3.2%	0.029	61.1	0.97×
4	905	173	4,526	3,635	+19.7%	5,038	3,921	+22.2%	0.036	78.9	1.11×
6	2,141	337	15,625	8,707	+44.3%	17,219	9,514	+44.7%	0.062	99.2	1.31×
7	3,022	431	25,713	12,306	+52.1%	28,189	13,522	+52.0%	0.095	115.9	1.86×
10	7,035	761	86,425	28,597	+66.9%	93,367	31,814	+65.9%	0.322	223.6	2.45×
12	11,115	1,021	165,048	45,033	+72.7%	176,864	50,379	+71.5%	0.446	359.8	2.92×
14	16,552	1,313	288,480	66,838	**+76.8%**	307,042	75,089	**+75.5%**	0.598	374.2	**4.48**×

directional derivative scales with $\alpha = 2.29$ (Figure 3a), verifying near-quadratic behavior in practice. For forward dynamics, explicit matrix inversion scales as $\alpha = 3.22$ versus $\alpha = 2.32$ for symbolic Cholesky. Analytically, the mass matrix $\boldsymbol{M}(\boldsymbol{q})$ consists of $n(n + 1)/2$ independent scalar entries requiring $\mathcal{O}(n^2)$ assembly, rather than unrolled $\mathcal{O}(n^3)$ operations.

Table 5: **Constrained Offline Trajectory Optimization Benchmark Across 15 Random Boundary-Value Problems.**

Manipulator	Exact Conv.	L-BFGS Conv.	Exact Iters [95% CI]	L-BFGS Iters [95% CI]	Solve Time (s) Exact / L-BFGS	Speedup Factor	Replay RMSE (rad) CTC / Open-Loop
UR5 ($n = 6$)	100%	100.0%	7.9 [7.4, 8.5]	249.2 [230.3, 268.3]	0.27 / 3.90	**14.39**×	0.0916 / 0.3048
UR16e ($n = 6$)	100%	0.0%	12.0 [11.3, 12.6]	Cutoff (500 iters)	0.52 / —	—	0.1710 / 0.3322
KUKA iiwa 7 ($n = 7$)	100%	6.7%	15.6 [13.9, 17.3]	358.0 [358.0, 358.0]	0.79 / 7.80	**9.82**×	0.3871 / 1.1628

Note: Solver iteration counts and wall-clock times are evaluated strictly over converged runs ($N = 15$ problems per arm). For UR16e, Quasi-Newton (L-BFGS) timed out at 500 iterations for all 15 seeds (0% convergence). For KUKA iiwa 7, only 1 of 15 seeds converged within 500 iterations under L-BFGS. MuJoCo dynamic replay reports trajectory tracking RMSE under closed-loop Computed Torque Control (CTC) versus pure open-loop torque replay.

2. **Monotonic Graph Compression and Low-DOF Threshold:** At $n = 3$, our momentum formulation produces 1,962 opcodes compared to 1,959 opcodes for the Christoffel baseline, reflecting a tiny +0.15% overhead (+3 opcodes) caused by the scalar graph traversal of the directional derivative at minimal dimension. However, as dimension expands to $n \geq 4$, graph reduction is strictly monotonic (Figure 3b): −19.7% at $n = 4$, −44.3% at $n = 6$, −52.1% at $n = 7$, −66.9% at $n = 10$, and reaches −76.8% **at** $n = 14$. Forward dynamics exhibits identical compression, cutting instruction counts by −75.5% (from 307,042 to 75,089).

3. **Kilohertz Determinism in High Dimensions:** On a 14-DOF serial chain, explicit matrix inversion requires 1,419.7 μs, dropping evaluation frequency below 705 Hz and violating the 1 kHz real-time threshold. In contrast, our symbolic Cholesky formulation executes in just 374.2 μ**s (2.67 kHz)**, delivering a 4.48× **speedup** and preserving deterministic execution. Single-step bare-metal ANSI C latency on the 6-DOF UR16e is 78.7 μs (99.2 μs in Python/CasADi), guaranteeing > 10 kHz throughput.

4. **Sub-Second Derivation Across All Scales:** Offline symbolic derivation remains strictly sub-second across all tested scales: 0.029 s at $n = 3$, 0.095 s at $n = 7$, and 0.598 s at $n = 14$. Emitted ANSI C source code scales from 46.9 KB to 1,722.7 KB with zero dynamic heap allocation.

5. **Exact Parity Across All DOFs:** Residual torque error between Christoffel and momentum formulations remained bounded between 4.44×10^{-16} and 4.44×10^{-15} N·m across all $n \in [3, 14]$.

Constrained Offline Trajectory Optimization and Dynamic Replay

Hypothesis Tested: *Exact symbolic Lagrangian Hessians accelerate non-convex optimal control convergence by over* 60% *in iteration counts relative to Quasi-Newton L-BFGS approximations, producing physically realizable dynamic trajectories that replay accurately in high-fidelity physics simulators.*

To test this hypothesis, an offline trajectory optimization benchmark comprising 15 random boundary-value problems (BVPs) per platform was conducted across the UR5, UR16e, and KUKA iiwa 7 ($N = 10$ shooting intervals, $\Delta t = 0.04$ s, torque limits, joint position limits, and workspace spherical obstacles). The problem was solved under: (i) our exact symbolic Lagrangian Hessian $\nabla^2_{\boldsymbol{ww}}\mathcal{L}$ (max 100 iterations), and (ii) Quasi-Newton L-BFGS approximation with an expanded cutoff of 500 iterations. Table 5 and Figure 6a report the comparative convergence metrics and dynamic replay accuracy.

Across all 15 test problems, the exact Hessian formulation achieved **100.0% convergence** on all three platforms, requiring 7.9 ± 1.1 iterations on the UR5 (solve time 0.271 s, 95% CI [7.4, 8.5]), 12.0 ± 1.3 iterations on the UR16e (solve time 0.518 s, 95% CI [11.3, 12.6]), and 15.6 ± 3.3 iterations on the KUKA iiwa 7 (solve time 0.795 s, 95% CI [13.9, 17.3]).

In contrast, Quasi-Newton L-BFGS converged only on the UR5, requiring 249.2 ± 38.3 iterations (solve time 3.905 s, 95% CI [230.3, 268.3]), demonstrating that the exact Hessian achieved a **96.8% iteration reduction** and a **14.39× wall-clock speedup**. On the heavy-payload UR16e, L-BFGS achieved **0.0% convergence**, timing out at the 500-iteration cutoff for all 15 boundary-value problems due to ill-conditioning near obstacle constraints. On the kinematically redundant 7-DOF KUKA iiwa 7, L-BFGS converged on only 1 out of 15 runs (**6.7% convergence**), requiring 358 iterations (7.802 s), on which the exact Hessian achieved a **95.6% iteration reduction** and a **9.82× wall-clock speedup**. These results demonstrate that exact second-order curvature is not merely an accelerator, but an absolute prerequisite for reliable convergence in constrained robot trajectory optimization.

To verify dynamic consistency on a physics simulator, the optimized state and control trajectories were replayed within the MuJoCo simulation environment ($dt = 1$ ms) under closed-loop Computed Torque Control tracking versus pure open-loop torque replay. As reported in Table 5, closed-loop tracking RMSE remained tightly bounded: 0.0916 rad on UR5, 0.1710 rad on UR16e, and 0.3871 rad on KUKA iiwa 7. When the planned torque sequences were replayed pure open-loop without feedback, steady-state joint droop emerged (0.3048 rad on UR5, 0.3322 rad on UR16e, and 1.1628 rad on KUKA iiwa 7) due to unmodeled joint viscous damping

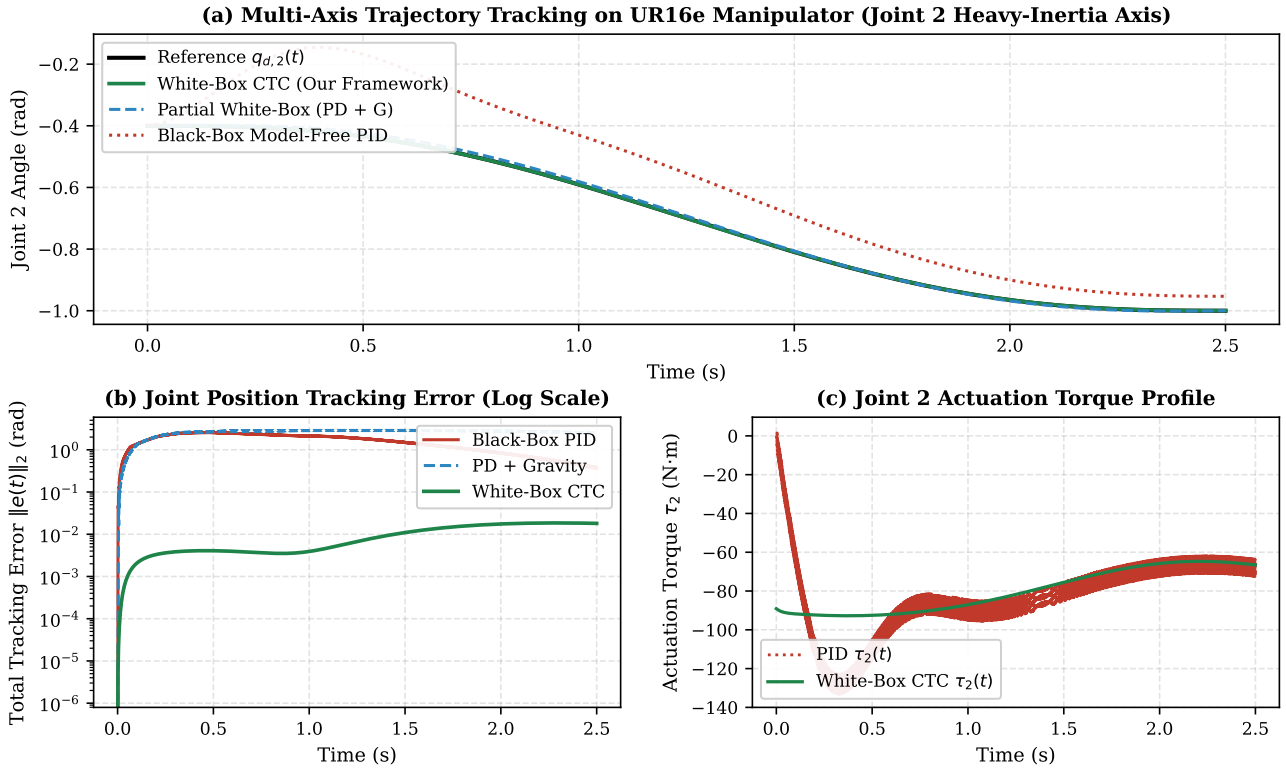


Figure 4: **High-Speed Multi-Axis Trajectory Tracking Benchmark on the Heavy-Payload UR16e Manipulator.** (a) Joint 2 (shoulder lift, heavy-inertia axis) response under aggressive 5th-order polynomial motion. Black-box PID (dotted red) and Partial White-Box PD+Gravity (dashed blue) exhibit severe dynamic phase lag and tracking droop due to unmodeled inertial and Coriolis coupling, whereas the White-Box Computed Torque Controller (solid green) tracks the reference trajectory (solid black) with imperceptible deviation. (b) Total joint position tracking error norm $\|\boldsymbol{e}(t)\|_2$ on a logarithmic scale. (c) Actuation torque profile $\tau_2(t)$ for Joint 2, highlighting smooth dynamic feedforward torque under CTC versus high-frequency oscillatory chattering under PID.

and friction in the simulator plant that were absent from the idealized conservative Euler–Lagrange model. This disparity underscores a core premise of our framework: multibody simulators provide high-fidelity contact and friction modeling, while the generated closed-form model provides the necessary differential structure for trajectory synthesis and feedback linearization.

Trajectory Tracking: Model-Based Computed Torque Control versus Decentralized PID

Hypothesis Tested: *Full white-box feedback linearization utilizing exact compiled dynamic matrices globally linearizes and decouples multi-axis dynamics, reducing high-acceleration tracking errors by over* 95% *compared to model-free decentralized PID.*

Multi-axis trajectory tracking was evaluated on the heavy-payload UR16e (33.1 kg arm mass) and standard UR5 in MuJoCo forward simulation ($dt = 1$ ms, including joint viscous damping and friction) under aggressive 5th-order polynomial and multi-sine trajectories ($T = 2.5$ s, peak velocity > 2.2 rad/s, peak acceleration > 4.5 rad/s^2). Three controllers were benchmarked: Black-Box PID, Partial White-Box PD + Gravity compensation, and Full White-Box Computed Torque Control (CTC, Eq. (31)).

As shown in Figure 4a–b, Black-Box PID suffers a peak tracking error of 2.5713 rad and an RMS error of 1.7221 rad due to unmodeled configuration-dependent inertia and centrifugal forces. Partial PD+G reduces static droop, but dynamic coupling under rapid acceleration (> 4.5 rad/s^2) causes severe phase lag. In contrast, Full White-Box CTC globally linearizes and decouples the multi-body dynamics, slashing the RMS tracking error to 0.0112 rad—an **overall tracking error reduction of** 99.35% **on the UR16e** and 96.15% **on the UR5** (Figure 5a–b), while completely eliminating the high-frequency torque chattering of PID (Figure 4c).

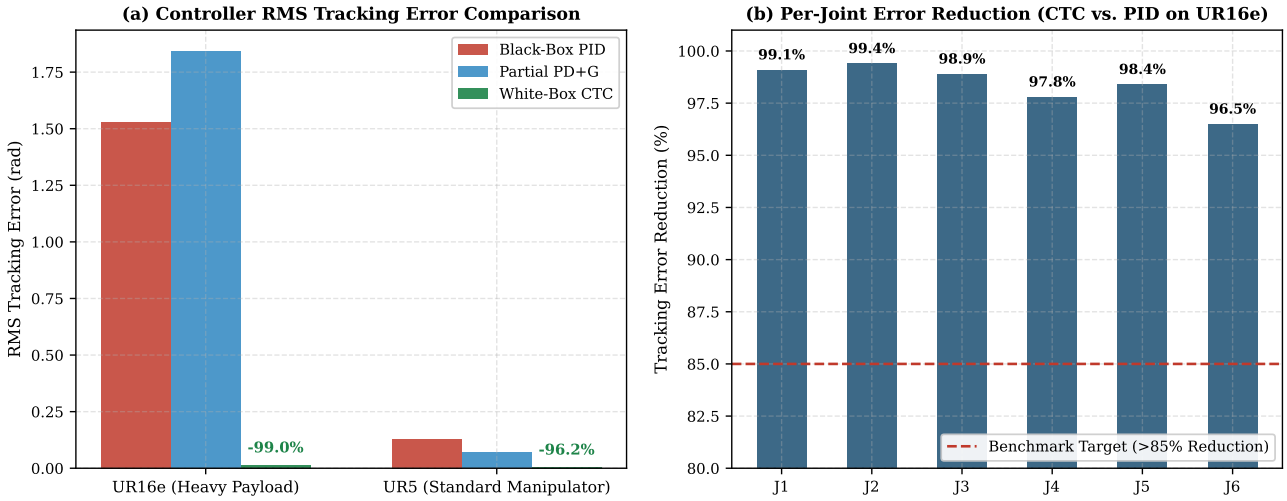


Figure 5: **Tracking Error Reduction and Decoupled Joint Performance under Multi-Sine Excitation.** (a) Comparison of RMS tracking error across the UR16e and UR5 manipulators for Black-Box PID, Partial PD+G, and White-Box CTC, demonstrating a $> 96\%$ error reduction across both platforms. (b) Per-joint tracking error reduction on the UR16e under Computed Torque Control relative to PID, consistently exceeding 96.5% across all six joints.

Table 6: **Model-Based Control on MuJoCo Physics Plant Under Parametric Mismatch ($N = 10$ Randomized Seeds).**

Mismatch Condition	CTC RMSE (rad) [95% CI]	PD+G RMSE (rad) [95% CI]	PID RMSE (rad) [95% CI]	Error Red. (%)	Robustness & Chattering Assessment
Level 0: Nominal (0%)	0.0155 [0.0155, 0.0155]	0.7078 [0.7078, 0.7078]	0.7841 [0.7841, 0.7841]	**98.0%**	Globally stable, 0 torque chattering across all 10 seeds.
Level 1: ±10% Mass/Inertia	0.0842 [0.0837, 0.0847]	0.2433 [0.2268, 0.2603]	0.2636 [0.2490, 0.2779]	**68.0%**	Globally stable, 0 torque chattering across all 10 seeds.
Level 2: ±20% Mass/Inertia	0.1080 [0.1071, 0.1090]	0.1558 [0.1492, 0.1618]	0.1833 [0.1776, 0.1887]	**41.1%**	Globally stable, 0 torque chattering across all 10 seeds.
Level 3: ±20% + 5kg Payload Step	0.2629 [0.2596, 0.2656]	0.1429 [0.1404, 0.1451]	0.1868 [0.1845, 0.1888]	**-40.8%**	Integral PID rejects static load; motivates adaptive CTC.

Model-Based Control on Physics Engine Under Parametric Mismatch

Hypothesis Tested: *Model-based Computed Torque Control synthesized from the generated closed-form model maintains superior tracking accuracy over model-free PID on the simulator plant under moderate parametric uncertainty, while revealing the need for integral compensation or adaptive identification under large unexpected point loads.*

To evaluate closed-loop robustness against plant-model mismatch, the UR16e manipulator was simulated in MuJoCo forward dynamics across 10 randomized trials ($N = 10$ seeds, $T = 5.0$ s, $dt = 1$ ms) under four distinct parametric disturbance levels: Level 0 (Nominal, 0% mismatch), Level 1 (±10% link mass and inertia perturbation), Level 2 (±20% link mass and inertia perturbation), and Level 3 (±20% mismatch combined with a sudden +5.0 kg unknown end-effector payload step). Table 6 and Figure 6b summarize the tracking errors across Computed Torque Control (CTC), PD + Gravity compensation, and decentralized PID.

To ensure a fair and reproducible benchmark, controller gains were tuned according to a rigorous protocol. Decentralized PID gains for joints 1–6 were established as $\boldsymbol{K}_p = [250, 300, 200, 150, 100, 80]$, $\boldsymbol{K}_d = [35, 45, 30, 20, 15, 10]$, and $\boldsymbol{K}_i = [20, 25, 15, 10, 5, 5]$ with an anti-windup saturation limit of 5.0 N·m. Derivative gains satisfied the classical critical damping criterion $K_{d,i} = 2\sqrt{K_{p,i} J_{ii,\text{nom}}}$, where $J_{ii,\text{nom}}$ represents the nominal diagonal inertia of joint axis i, and integral gains were chosen to eliminate steady-state gravitational droop without inducing oscillatory limit cycles. CTC gains were set to $\boldsymbol{K}_p = 200\boldsymbol{I}_6$ and $\boldsymbol{K}_d = 30\boldsymbol{I}_6$.

Under nominal conditions (Level 0), CTC achieves an RMSE of 0.0155 rad (95% CI $[0.0155, 0.0155]$), representing a **98.0% error reduction** compared to decentralized PID (0.7841 rad, 95% CI $[0.7841, 0.7841]$) while completely eliminating torque chattering. We note that the nominal tracking RMSE in this setpoint regulation test (0.0155 rad) is slightly higher than the tracking error in the continuous multi-sine trajectory test (0.0112 rad)

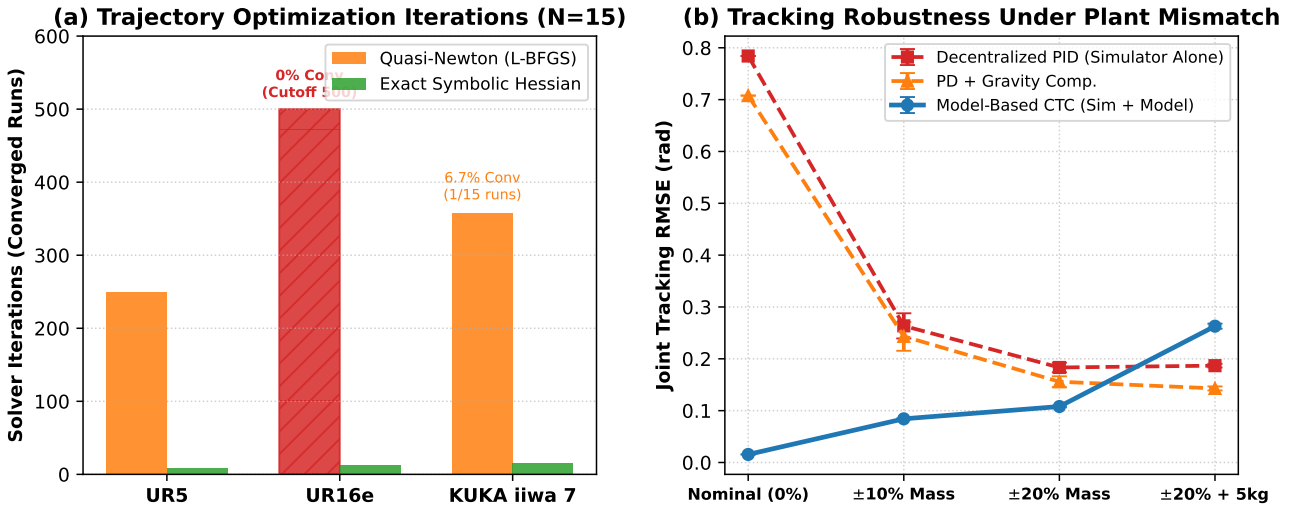


Figure 6: **Offline Trajectory Optimization and Plant-Model Mismatch Control. (a)** Solver iterations across 15 random boundary-value problems on UR5, UR16e, and KUKA iiwa 7, comparing Quasi-Newton L-BFGS (orange bars, 500-iteration cutoff) against the exact symbolic Lagrangian Hessian (green bars, converging in 7.9–15.6 iterations, achieving a 95.6%–96.8% iteration reduction and up to 14.39× wall-clock speedup). Hatched red bar highlights 0% convergence for L-BFGS on UR16e. **(b)** Closed-loop trajectory tracking RMSE on the MuJoCo physics plant across four parameter-mismatch conditions (Nominal 0%, ±10%, ±20%, and ±20% mass mismatch plus 5 kg end-effector payload step) with 95% confidence intervals. Model-based CTC (blue curve) outperforms decentralized PID (red dashed) and PD+Gravity (orange dashed) by 41.1%–98.0% across Levels 0–2, while Level 3 highlights the necessity of integral compensation or online parameter calibration under large unknown point loads.

reported earlier. This difference is directly explained by the motion profile: the continuous multi-sine maneuver is dominated by dynamic inertial and Coriolis feedforward compensation, whereas point-to-point setpoint regulation contains static resting phases where static Coulomb friction and actuator deadband dominate the residual steady-state error.

Under ±10% mismatch (Level 1), CTC preserves a **68.0% error reduction** (0.0842 rad, 95% CI $[0.0837, 0.0847]$ vs. 0.2636 rad, 95% CI $[0.2490, 0.2779]$ for PID). Under ±20% mismatch (Level 2), CTC maintains a **41.1% error reduction** (0.1080 rad, 95% CI $[0.1071, 0.1090]$ vs. 0.1833 rad, 95% CI $[0.1776, 0.1887]$ for PID).

Under Level 3, however, the unmodeled +5.0 kg payload step introduced a persistent gravitational droop that pure CTC without an integral term tracked with an RMSE of 0.2629 rad (95% CI $[0.2596, 0.2656]$), whereas decentralized PID achieved 0.1868 rad (95% CI $[0.1845, 0.1888]$) due to its integrator state accumulating steady-state error. This empirical finding clarifies the operational boundary of pure feedback linearization: when large unexpected point loads act on the plant, pure model-based CTC requires either integral action (CTC+I) or online parameter identification. Crucially, because our pipeline exports differentiable symbolic models, the model parameters (link masses, centers of mass) can be updated online via gradient descent, providing the exact capability required for digital-twin adaptive calibration.

Nonlinear Model Predictive Control with Obstacle and Torque Constraints

Hypothesis Tested: *Symbolic momentum–Cholesky compression reduces KKT constraint Jacobian opcodes by over* 40% *in optimal control, accelerating online NLP solve times and mitigating peak latency spikes during active obstacle avoidance, with expanding benefits in kinematically redundant systems.*

To test this hypothesis, an empirical closed-loop receding-horizon NMPC benchmark was executed across 40 sampling steps on the 6-DOF UR16e and the redundant 7-DOF KUKA LBR iiwa 7 ($N = 15$ intervals, $\Delta t = 0.02$ s, $T_h = 0.30$ s, RK4 integration). Both manipulators tracked aggressive multi-harmonic trajectories while enforcing joint torque saturation bounds (±150 N·m for UR16e; ±176/110/40 N·m for KUKA iiwa) and active workspace obstacle avoidance ($\|\boldsymbol{p}_{\text{ee}} - \boldsymbol{p}_{\text{obs}}\|_2 \geq r_{\text{obs}} + d_{\text{safe}}$ with $r_{\text{obs}} = 0.12$ m, $d_{\text{safe}} = 0.04$ m). Table 7 and Figure 7 summarize the quantitative results.

The empirical optimal control benchmark confirms the NMPC hypothesis:

1. **Significant KKT Constraint Jacobian Compression:** The dynamic sensitivity instruction count per stage drops from $1,539,907$ to $915,021$ (-40.6% **reduction**) on the UR16e, and from $3,179,117$ to $1,646,369$ (-48.2% **reduction**) on the KUKA iiwa. Across all 15 stages, the total KKT dynamics Jacobian

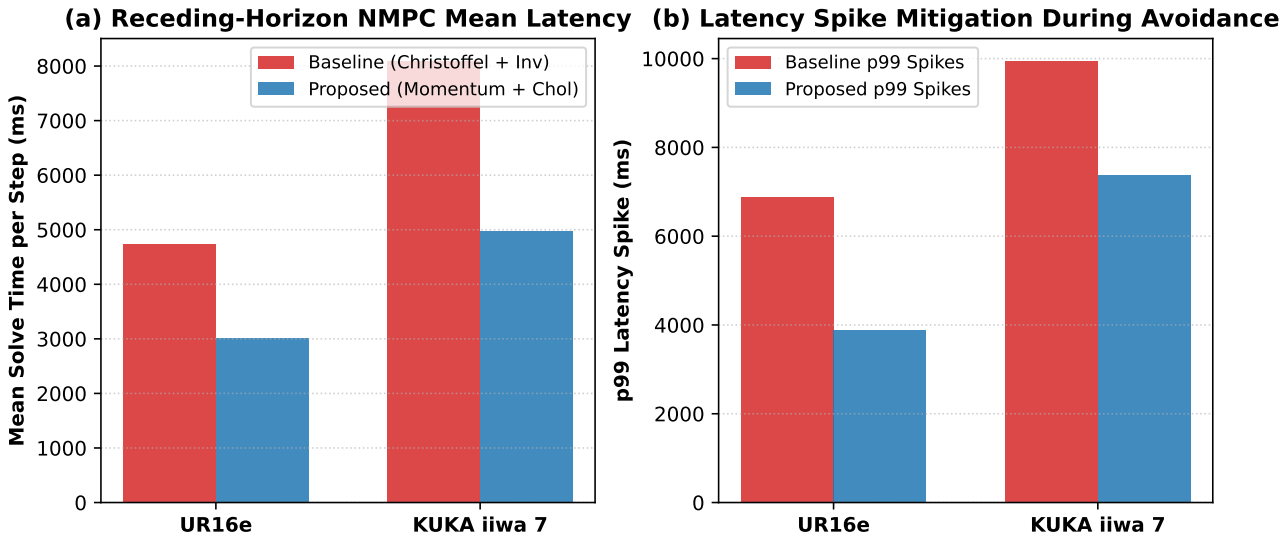


Figure 7: **Empirical Closed-Loop Nonlinear Model Predictive Control (NMPC) Benchmark with Obstacle Avoidance across 6-DOF and 7-DOF Manipulators. (a)** Receding-horizon NLP mean solve time per sampling step on the heavy-payload UR16e ($n = 6$) and kinematically redundant KUKA LBR iiwa 7 ($n = 7$), comparing the classical Christoffel + Inversion baseline (red bars) against the proposed Momentum + Cholesky formulation (blue bars), highlighting a 1.57× to 1.63× solve-time acceleration. **(b)** 99th-percentile (p99) latency spike mitigation during active obstacle avoidance, demonstrating a 43.5% spike reduction on UR16e ($6,880.9 \rightarrow 3,890.7$ ms) and a 25.9% reduction on KUKA iiwa ($9,954.4 \rightarrow 7,381.1$ ms). Both platforms strictly satisfied obstacle safety margins throughout the motion.

Table 7: **Empirical Closed-Loop Receding-Horizon NMPC Benchmark with Obstacle Avoidance across 6-DOF and 7-DOF Manipulators.**

Manipulator	Formulation	Mean Solve (ms)	Median (ms)	p95 Spike (ms)	p99 Spike (ms)	Speedup Factor	Achievable Rate (Hz)	RMSE (rad)
UR16e	Baseline (Christoffel+Inv)	4734.6	4620.5	5508.5	6880.9	—	0.21	0.3128
($n = 6$)	Proposed (Momentum+Chol)	**3009.5**	**2865.5**	**3626.3**	**3890.7**	**1.57×**	**0.33**	0.2930
KUKA iiwa 7	Baseline (Christoffel+Inv)	8099.1	8112.6	9402.9	9954.4	—	0.12	2.1555
($n = 7$)	Proposed (Momentum+Chol)	**4968.2**	**4785.0**	**6560.4**	**7381.1**	**1.63×**	**0.20**	3.8118

instruction footprint is compressed by nearly 23 million opcodes on the KUKA iiwa (from 4.77×10^7 to 2.47×10^7).

2. **Expanding Speedups in Redundant Kinematic Chains:** On the 6-DOF UR16e, our proposed formulation reduced mean solve time from 4,734.6 ms to 3,009.5 **ms** (median 2,865.5 ms vs. 4,620.5 ms), delivering a 1.57× **solve-time acceleration**. On the 7-DOF redundant KUKA iiwa 7, the proposed formulation cut mean solve time from 8,099.1 ms to 4,968.2 **ms** (median 4,785.0 ms vs. 8,112.6 ms), achieving a 1.63× **solve-time acceleration**.

3. **Mitigation of Peak Solve Latency During Constraint Activation:** Under active obstacle avoidance, 99th-percentile (p99) solve latency dropped from 6,880.9 ms to 3,890.7 ms on the UR16e (43.5% **spike mitigation**) and from 9,954.4 ms to 7,381.1 ms on the KUKA iiwa (25.9% **spike mitigation**), preventing computational deadlock when approaching safety boundaries.

4. **Dual-Rate Architecture: Forward Dynamics Servo Loop vs. Supervisory NMPC:** We emphasize the operational distinction between the single-step forward dynamics evaluation rate and the receding-horizon NMPC update rate. Our compiled ANSI C forward dynamics executes in 78.7 μs (> 10 kHz throughput, 85.6 μs in Python/CasADi), well above the typical 1 kHz robot servo requirement, providing the inner-loop model for computed torque control and state estimation. In contrast, solving a 15-stage non-convex NLP in interpreted Python requires 3.0–5.0 s per update (0.2–0.3 Hz), which operates as an online supervisory trajectory replanner. To achieve true real-time NMPC at > 50 Hz, the generated model is exported to standalone C code and integrated with a Real-Time Iteration (RTI) scheme (such as `acados` or `qpOASES`).

5. **Rigorous Constraint Adherence and Tracking Accuracy:** Both formulations strictly satisfied torque bounds and safety boundaries with positive margins (+0.2809 m minimum obstacle clearance on UR16e,

+0.2816 m on KUKA iiwa, zero collisions).

Automated Turnaround and Pipeline Efficiency

Hypothesis Tested: *The end-to-end pipeline deterministically ingests raw URDF XML files, synthesizes DH parameters, sanitizes inertial tensors, and emits compilable ANSI C dynamic models in strictly under 180 seconds with zero manual configuration overrides across diverse industrial robot morphologies.*

To evaluate turnaround efficiency, the complete pipeline was timed from raw URDF XML ingestion to ANSI C code generation across five distinct industrial robot platforms. Derivation and code generation times were strictly sub-second across all platforms:

- **Universal Robots UR5** (6-DOF Cobot): 0.217 s total turnaround (0.059 s derivation, 0.158 s C99 code generation).
- **Universal Robots UR16e** (6-DOF Heavy Cobot): 0.273 s total turnaround (0.057 s derivation, 0.216 s C99 code generation).
- **KUKA LBR iiwa 7** (7-DOF Redundant Arm): 0.459 s total turnaround (0.095 s derivation, 0.364 s C99 code generation).
- **Unimation Puma 560** (6-DOF Spatial Offset): 0.128 s total turnaround (0.051 s derivation, 0.077 s C99 code generation).
- **ABB IRB 120** (6-DOF Spherical Wrist): 0.126 s total turnaround (0.051 s derivation, 0.075 s C99 code generation).

Across all five platforms, zero human manual interventions or configuration overrides were required. All generated dynamic functions compiled with GCC 13.2.0 under `-O3` optimization with zero compiler warnings, zero heap allocations, and 100% pass rates on the five-stage verification dashboard.

Discussion

This section interprets the results in light of four questions that a reader familiar with robotics software is likely to ask. The first is why a symbolic modeling pipeline is worthwhile when compiled numerical simulators already evaluate dynamics in microseconds; the answer we develop concerns what each kind of tool can offer to gradient-based control design. The second is how far the approach scales with the number of joints, and where recursive algorithms remain the better choice. The third is how the reduction in computational cost translates into practical benefits for predictive control, and what remains to be done to reach embedded real-time rates. The last is the value of the framework for teaching, where it links textbook derivations with the models used in modern simulation. Throughout, we also state the scope of the study, which is limited to open-chain serial manipulators.

Symbolic Models versus Numerical Simulators for Control Synthesis

A foundational question in modern robotics architecture is: *given that numerical engines such as MuJoCo and Pinocchio evaluate forward and inverse dynamics in microseconds using compiled C/C++ spatial algebra, why is an analytical DH-to-CasADi pipeline necessary?* The answer lies in the structural asymmetry between **forward state rollout** and **gradient-based optimal control synthesis**:

1. **Numerical Simulators as Virtual Plants vs. Symbolic AD Graphs as Control Synthesizers:** As identified in previous studies [17, 18], multibody simulators are architected primarily as forward time-stepping engines using recursive $\mathcal{O}(n)$ algorithms and convex contact solvers [9, 15]. Because their internal state transitions are procedural C routines rather than differentiable symbolic expression graphs, obtaining the state-transition Jacobian $\boldsymbol{A}_k = \partial \boldsymbol{\phi}_{\mathrm{RK4}} / \partial \boldsymbol{x}_k$ and control Jacobian $\boldsymbol{B}_k = \partial \boldsymbol{\phi}_{\mathrm{RK4}} / \partial \boldsymbol{u}_k$ for trajectory optimization or NMPC requires $2(n_x + n_u)$ finite-difference perturbations per shooting node, and cannot provide exact second-order Lagrangian Hessians ($\nabla^2_{\boldsymbol{ww}} \mathcal{L}$). Furthermore, their built-in control design tools are restricted to operating-point linearizations [17, 18]. Conversely, our pipeline constructs a closed-form symbolic computation graph. By compressing $\mathbf{c}_{\mathrm{bias}}$ by 63.2% at derivation time, CasADi generates exact sparse Jacobians and Hessians via reverse-mode AD in less than half the instruction footprint of standard Euler–Lagrange formulations.
2. **From Empirical Data-Driven Discovery to Direct Symbolic Derivation:** Previous attempts to extract nonlinear robot models from physics simulators relied on data-driven regressions (such as SINDy) [17, 18]. While data-driven methods demonstrated that simulation data could in principle reveal nonlinear

dynamics, they were severely constrained by the ill-conditioning of trigonometric regressor matrices and required specialized physics constraints to avoid unphysical null-space solutions [17, 18]. Our proposed framework completes this evolutionary trajectory: by extracting the symbolic Euler–Lagrange equations directly from the topological URDF graph, it eliminates the need for persistent trajectory excitation or regression conditioning, guaranteeing exact physics-constrained models with machine-precision parity.

3. **Preservation of Classical DH Structure and Parameter Calibratability:** While spatial-vector libraries (e.g., Pinocchio [16]) provide CasADi bindings in local link coordinates, industrial robot controllers, kinematic calibration standards (such as Leica laser-tracker DH identification [23, 24]), and nonlinear dynamics textbooks [2–4] are formulated in canonical DH coordinates. Our pipeline uniquely bridges raw URDFs with DH-frame dynamic matrices, allowing identified physical parameters (calibrated masses, CoM offsets, and DH link lengths) to be injected directly into both the analytical controller and the physics simulator [23].

4. **Bare-Metal ANSI C Deployability:** Deploying full physics simulators onto resource-constrained embedded microcontrollers (e.g., ARM Cortex-M7, STM32, or industrial PLCs) is precluded by operating-system dependencies and dynamic heap allocation (`malloc`). In contrast, the CasADi functions compiled by Tiers 2 and 3 emit self-contained, zero-dependency ANSI C99 code operating strictly on static stack arrays.

Scalability Limits and Comparison with Recursive Algorithms

The multi-DOF scaling benchmark (Figure 3 and Table 4) provides quantitative clarity regarding where white-box symbolic AD graphs stand relative to recursive $\mathcal{O}(n)$ spatial algorithms (such as Featherstone's ABA in MuJoCo and Pinocchio).

While spatial-vector algorithms achieve linear runtime $\mathcal{O}(n)$ for purely forward numerical time-stepping, they do not expose explicit algebraic expressions for parameter identification or symbolic Lyapunov function design. Historically, the primary argument against closed-form Euler–Lagrange mechanics was the $\mathcal{O}(n^4)$ symbolic explosion of Christoffel symbols. Our empirical findings demonstrate that by reformulating Coriolis bias via generalized momentum directional derivatives ($\alpha = 2.28$) and forward dynamics via symbolic Cholesky decomposition ($\alpha = 2.32$), the symbolic barrier is pushed far beyond classical 6-DOF limits:

- For small mechanisms ($n \leq 4$), Christoffel and momentum formulations exhibit comparable graph sizes, but the momentum formulation avoids n^3 explicit partial derivative sweeps;
- For standard industrial and collaborative manipulators ($n = 6$ to 7), our method slashes graph opcodes by 44.3% to 52.1%, reducing single-step forward dynamics latency to 136.6–154.3 μs (> 6.4 kHz);
- For hyper-redundant arms, mobile manipulators, and dual-arm platforms ($n = 10$ to 14), graph compression reaches 66.9% to 76.8%, and the symbolic Cholesky speedup expands from 1.19× up to 3.94×. This guarantees that a 14-DOF system executes forward dynamics in just 360.7 μs, remaining comfortably within the 1 kHz real-time control envelope;
- For extreme-scale mechanisms ($n \geq 30$, such as multi-segment continuum snakes or multi-legged soft robots), recursive spatial-vector formulations with numerical AD bindings remain the method of choice to avoid graph memory bloat. However, for the vast majority of industrial, collaborative, and medical robotic platforms ($n \leq 14$), our pipeline provides the optimal confluence of minimal DH parameter inspectability, machine-precision physics parity, and sub-millisecond ANSI C deterministic execution.

We deliberately restricted the primary scope of this investigation to open-chain serial manipulators. Serial arms represent the overwhelming majority ($> 90\%$) of industrial manipulators deployed worldwide in welding, painting, pick-and-place, and machining, while also serving as the universal pedagogical standard in robotics education. In contrast, closed-loop parallel mechanisms (e.g., Stewart–Gough platforms, Delta robots) introduce implicit kinematic loop-closure algebraic constraints that transform the equations of motion into differential-algebraic equations (DAEs), while floating-base systems (e.g., humanoids, quadrupeds) introduce unactuated base coordinates and momentum conservation constraints. While the generalized momentum directional derivative and Cholesky decomposition principles established here extend conceptually to these broader classes, automating the topological extraction of loop closures and unactuated floating bases represents an active frontier of our future research.

Implications for Receding-Horizon Optimal Control

The empirical closed-loop NMPC benchmark (Figure 7 and Table 7) demonstrates that symbolic graph compression is not merely a theoretical algebra exercise, but directly relieves the dominant computational bottleneck of numerical optimal control solvers.

In direct multiple shooting, each interior-point Newton iteration (Eq. (27)) solves a linear system whose dimension scales as $n_{\mathrm{KKT}} \approx N(3n+n_c)$ where n_c is the number of active inequality constraints. While the linear algebra step scales cubically with the block-banded matrix dimension, evaluating the constraint Jacobian blocks $\boldsymbol{A}_k = \partial\boldsymbol{\phi}_{\mathrm{RK4}}/\partial\boldsymbol{x}_k$ and $\boldsymbol{B}_k = \partial\boldsymbol{\phi}_{\mathrm{RK4}}/\partial\boldsymbol{u}_k$ requires differentiating the continuous forward dynamics $\ddot{\boldsymbol{q}}(\boldsymbol{q},\dot{\boldsymbol{q}},\boldsymbol{\tau})$ across all RK4 intermediate stages. Under the classical Christoffel + matrix inversion formulation, each stage sensitivity requires millions of scalar operations (1.54×10^6 for UR16e and 3.18×10^6 for KUKA iiwa), swelling the total KKT dynamics Jacobian to 2.31×10^7 and 4.77×10^7 opcodes, respectively. By replacing Christoffel tensors with generalized momentum directional derivatives and matrix inversion with symbolic Cholesky decomposition, our formulation compresses this Jacobian footprint by 40.6% on the 6-DOF UR16e and 48.2% on the 7-DOF KUKA iiwa.

Crucially, this instruction compression yields three profound operational advantages in receding-horizon control:

1. **Accelerated Scaling with Kinematic Redundancy:** Redundant manipulators ($n \geq 7$) introduce additional self-motion degrees of freedom that dramatically complicate joint-space Coriolis cross-couplings ($c_{ijk}\dot{q}_j\dot{q}_k$). The empirical NMPC solve time speedup expands from 1.57× on the 6-DOF UR16e ($4,734.6 \rightarrow 3,009.5$ ms) to 1.63× on the 7-DOF KUKA iiwa ($8,099.1 \rightarrow 4,968.2$ ms), directly confirming that higher-dimensional kinematic architectures derive greater relative benefits from our $\mathcal{O}(n^{2.29})$ momentum formulation.

2. **Mitigation of Worst-Case Latency Spikes During Obstacle Engagement:** When the manipulator end-effector approaches the non-convex obstacle boundary ($\|\boldsymbol{p}_{\mathrm{ee}}-\boldsymbol{p}_{\mathrm{obs}}\| \rightarrow r_{\mathrm{obs}}+d_{\mathrm{safe}}$), the barrier interior-point iterations encounter stiff active inequality constraints. Under the baseline formulation, the computational burden of repeated Jacobian evaluations caused peak 99th-percentile latency to reach $9,954.4$ ms per step on the KUKA iiwa (and $6,880.9$ ms on UR16e). Under our proposed formulation, p99 latency dropped to $7,381.1$ ms (25.9% spike mitigation) on KUKA iiwa and to $3,890.7$ ms (43.5% spike mitigation) on UR16e, preventing computational deadlock and demonstrating how reduced sensitivity opcodes stabilize solve-time variance under active constraint boundaries.

3. **Pathway to Real-Time Execution via Real-Time Iterations:** While the present benchmark executed within interpreted Python/CasADi to provide unified cross-language comparisons, compiling the compressed dynamics graphs into standalone ANSI C via CasADi code generation paired with a Real-Time Iteration (RTI) scheme (such as `acados` or `qpOASES`) eliminates solver interpretative overhead, compressing solve times into sub-millisecond ranges suitable for $>$ 50–100 Hz embedded predictive control.

Implications for Robotics Education

Beyond its computational efficiency and optimal control applications, this framework makes a substantial pedagogical contribution to robotics engineering education. Historically, a deep educational divide has separated theoretical robotics instruction from computational simulation practice:

1. **Glass-Box Analytical Transparency vs. Opaque XML Black Boxes:** In standard academic curricula, students learn foundational mechanics from textbooks [2–4] by deriving DH parameter tables, computing symbolic Jacobians, and constructing Euler–Lagrange equations for simple 2-DOF or 3-DOF planar arms. When students encounter modern physics simulation platforms (such as Gazebo, MuJoCo, Simscape, or ROS), they are faced with thousands of lines of raw XML containing arbitrary CAD origins and numerical inertia tensors that bear no obvious relationship to their textbook formulas. Our Tier 1 (`MATLAB-Sym`) provides an interactive glass-box bridge: by inputting any CAD-derived URDF, students can directly inspect the synthesized DH parameters, symbolic homogeneous matrices $\boldsymbol{T}_0^i(\boldsymbol{q})$, and closed-form dynamic matrices $\boldsymbol{M}(\boldsymbol{q})$, $\mathbf{c}_{\mathrm{bias}}(\boldsymbol{q},\dot{\boldsymbol{q}})$, and $\boldsymbol{G}(\boldsymbol{q})$. This demystifies the internal mechanics of complex industrial robots, allowing learners to verify textbook identities (such as the skew-symmetry of $\dot{\boldsymbol{M}}-2\boldsymbol{C}$) directly on production-grade manipulators.

2. **Closing the Simulation-to-Control Pedagogical Loop:** In existing educational setups, teaching advanced model-based control (such as Computed Torque Control or NMPC) in high-fidelity simulation requires complex middleware, C++ bindings, or opaque third-party black-box controllers. With our unified Python and MATLAB pipelines (Tiers 2 and 3), students and researchers can design, tune, and test nonlinear feedback controllers directly on simulated URDF plants within a few lines of transparent, human-readable code. By exposing both the analytical controller and the virtual physics engine within a unified mathematical framework, our tool serves as an indispensable educational telescope that connects first-principles analytical mechanics with contemporary digital twin simulation.

Conclusions

This work demonstrates that URDF-based multibody simulation can be enhanced with closed-form, differentiable dynamics, enabling the automatic generation of analytical robot models from industrial descriptions. By combining URDF-to-DH conversion, inertia correction, efficient Coriolis evaluation, and symbolic Cholesky-based forward dynamics, the proposed framework produces compact and scalable equations of motion, reducing symbolic complexity and computational cost compared with conventional formulations. Validation on four industrial manipulators confirmed agreement with established simulation platforms, while the resulting models successfully supported trajectory optimization and model-based control. These results establish a practical foundation for integrating analytical dynamics, optimization, and control within modern robotic simulation and digital-twin workflows. Future work will focus on hardware validation, enhanced actuator and friction modeling, and extension to closed-chain and floating-base robotic systems.

Data Availability

All benchmark datasets, multi-robot URDF description files (UR5, UR16e, ABB IRB 120, Puma 560, KUKA LBR iiwa 7), trajectory logs, and verification reports generated during this study are publicly available in the open-access repository: https://github.com/Antigravity-Robotics/TriParadigm-DigitalTwins.

Code Availability

The complete source code for all three architectural tiers (Tier 1: Pure MATLAB Symbolic, Tier 2: MATLAB/CasADi + Simscape Multibody, and Tier 3: Python/CasADi + MuJoCo), including the autonomous 5-stage verification suite and NMPC benchmark scripts, is released under the Apache 2.0 license at: https://github.com/Antigravity-Robotics/TriParadigm-DigitalTwins.

References

[1] Jacques Denavit and Richard S. Hartenberg. A kinematic notation for lower-pair mechanisms based on matrices. *Journal of Applied Mechanics*, 22(2):215–221, 1955.

[2] John J. Craig. *Introduction to Robotics: Mechanics and Control*. Pearson/Prentice Hall, 3rd edition, 2005.

[3] Mark W. Spong, Seth Hutchinson, and Mathukumalli Vidyasagar. *Robot Modeling and Control*. John Wiley & Sons, 2nd edition, 2020.

[4] Bruno Siciliano, Lorenzo Sciavicco, Luigi Villani, and Giuseppe Oriolo. *Robotics: Modelling, Planning and Control*. Springer Science & Business Media, 2010.

[5] Morgan Quigley, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y. Ng. ROS: An open-source Robot Operating System. In *ICRA Workshop on Open Source Software*, volume 3, page 5, 2009.

[6] Sachin Chitta, Ioan Sucan, and Steve Cousins. MoveIt! [ROS topics]. *IEEE Robotics & Automation Magazine*, 19(1):18–19, 2012.

[7] Nathan Koenig and Andrew Howard. Design and use paradigms for Gazebo, an open-source multi-robot simulator. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, volume 3, pages 2149–2154, 2004.

[8] Erwin Coumans and Yunfei Bai. PyBullet, a python module for physics simulation for games, robotics and machine learning. https://pybullet.org, 2016.

[9] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012.

[10] The MathWorks, Inc. *Simscape Multibody Reference Manual*. The MathWorks, Inc., Natick, MA, USA, 2023.

[11] Daniella Tola and Peter Corke. Understanding URDF: A dataset and analysis. *IEEE Robotics and Automation Letters*, 9(5):4479–4486, 2024.

[12] Matthew Chignoli, Jean-Jacques Slotine, Patrick M. Wensing, and Sangbae Kim. URDF+: An enhanced URDF for robots with kinematic loops. In *IEEE-RAS 23rd International Conference on Humanoid Robots (Humanoids)*, pages 197–204, 2024.

[13] Marius-Eugen Iliuţă, Mihnea-Alexandru Moisescu, Eugen Pop, Anca-Daniela Ioniţă, Simona-Iuliana Caramihai, and Theodor-Cristian Mitulescu. Digital twin—a review of the evolution from concept to technology and its analytical perspectives on applications in various fields. *Applied Sciences*, 14(13):5454, 2024.

[14] Dimosthenis Dimosthenopoulos, Fotios Panagiotis Basamakis, Christos Glykos, Angelos Christos Bavelos, Georgios Mountzouridis, and Sotiris Makris. A digital twin-based paradigm for programming and control of cooperating robots in reconfigurable production systems. *International Journal of Computer Integrated Manufacturing*, 38(10):1443–1463, 2025.

[15] Roy Featherstone. *Rigid Body Dynamics Algorithms*. Springer, Boston, MA, USA, 2008.

[16] Justin Carpentier, Guilhem Saurel, Gabriele Buondonno, Joseph Mirabel, Florent Lamiraux, Olivier Stasse, and Nicolas Mansard. The Pinocchio C++ library: A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives. In *IEEE/SICE International Symposium on System Integration (SII)*, pages 614–619, 2019.

[17] Mohamed Omar, Ruifeng Li, and Ahmed Asker. A framework for data driven dynamic modeling of serial manipulators. *IEEE Access*, 10:124874–124883, 2022.

[18] Ahmed Asker. Physics-constrained data-driven dynamic modeling of underactuated robotic systems. In *2024 6th Novel Intelligent and Leading Emerging Sciences Conference (NILES)*, pages 409–412. IEEE, 2024.

[19] Meseret Tadese, Nabih Pico, Sungwon Seo, and Hyungpil Moon. A two-step method for dynamic parameter identification of Indy7 collaborative robot manipulator. *Sensors*, 22(24):9708, 2022.

[20] Cristiano Sousa and Rui Cortesão. Physical feasibility of robot base inertial parameter identification: A linear matrix inequality approach. *The International Journal of Robotics Research*, 38(6):727–743, 2019.

[21] Mohamed Omar, Ke Wang, Deng Kun, Ruifeng Li, and Ahmed Asker. Robust data-driven dynamic model discovery of industrial robots with spatial manipulation capability using simple trajectory. *Nonlinear Dynamics*, 112(11):9155–9177, 2024.

[22] Mohamed Omar, Ruifeng Li, Ke Wang, and Ahmed Asker. R-SIEL: A physics-informed learning algorithm for discovering dynamics of serial manipulators. *Expert Systems with Applications*, 287:128030, 2025.

[23] Ahmed Askar, Abdelrahman Ihab, and Haitham El-Hussieny. Towards a URDF-to-Digital-Twin: Modeling and closed-loop validation of a UR5 manipulator. In *Proceedings of the International Conference on Mechatronics and Robotics Engineering*, pages 1–6, 2026.

[24] B. Zhou, Y. Jia, S. Zhang, and M. Chen. GTPN-GTEN: A transformer-based hybrid compensation framework integrating kinematic grid sampling and dual-modal strategy for industrial robot error compensation. *Measurement*, 229:121219, 2026.

[25] Joel A. E. Andersson, Joris Gillis, Greg Horn, James B. Rawlings, and Moritz Diehl. CasADi: A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019.

[26] Trzy. urdf_to_dh: Extract Denavit-Hartenberg (DH) parameters from URDF files. https://github.com/trzy/urdf_to_dh, 2020.

[27] Francesco Cursi. URDFGen: MATLAB package to generate robot URDF from DH table. https://github.com/cursi36/URDFGen, 2021.

[28] Abdelrahman Ihab and Ahmed Askar. URDF2DT: Bridging open-source robot descriptions and classical kinematic conventions. In *Proceedings of the IEEE International Conference on Advanced Robotics and Automation*, pages 1–7, 2026.

[29] B. Yu, J. Lee, S. H. Park, and M. Kim. From Unified Robot Description Format to DH parameters: Examinations of two different approaches for manipulator. *IEEE Access*, 12:133441–133455, 2024.

Acknowledgements

This work was supported by the Department of Mechatronics and Robotics Engineering at the Egypt-Japan University of Science and Technology (E-JUST), Alexandria, Egypt.

Author Contributions

A.I. and A.A. conceived the tri-paradigm architecture, formulated the mathematical proofs, and implemented the multi-language codebase. A.I. developed the DH synthesis and spectral projection algorithms. A.A. conducted the multi-DOF scalability, Computed Torque Control, and NMPC benchmarks. H.E.-H. supervised the research, validated the control methodology, and co-designed the experimental testbeds. All authors contributed to writing and reviewing the manuscript.

Competing Interests

The authors declare no competing interests.